COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Mathematics for Games and Computing

Student handbook / expanded Volume 02

20 lessons · Modules 06-10 · Worked derivations · Runnable examples · 100 lesson questions

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Contents

PDF bookmarks also navigate the lessons, quizzes, assignments and references.

Mathematics for Games and Computing 4

Module 06 — Discrete mathematical reasoning 6

Lesson 06.1 — Algebra, functions, logarithms and trigonometry . . . 7

Lesson 06.2 — Logic, sets and quantified claims . . . 10

Lesson 06.3 — Proofs, induction and invariants . . . 13

Lesson 06.4 — Counting, graphs and recurrences . . . 16

Module 07 — Linear algebra and game geometry 19

Lesson 07.1 — Vectors, dot products and projections . . . 20

Lesson 07.2 — Matrices, bases and linear systems . . . 24

Lesson 07.3 — Homogeneous transforms, cameras and quaternions . . . 27

Lesson 07.4 — Eigenvalues, least squares and singular values . . . 30

Module 08 — Calculus, mechanics and numerical simulation 33

Lesson 08.1 — Limits, derivatives and rates . . . 34

Lesson 08.2 — Integrals, accumulation and continuous probability . . . 37

Lesson 08.3 — Newtonian mechanics and differential equations . . . 40

Lesson 08.4 — Time integration, stability and error . . . 43

Module 09 — Probability, statistics and evidence 47

Lesson 09.1 — Probability and conditional reasoning . . . 48

Lesson 09.2 — Random variables, distributions and sampling . . . 51

Lesson 09.3 — Inference, uncertainty and benchmark design . . . 54

Lesson 09.4 — Regression, correlation and ethical telemetry 57

Module 10 — Signals, optimization and numerical foundations 60

Lesson 10.1 — Multivariable calculus and gradients 61

Lesson 10.2 — Sampling, aliasing and Fourier ideas 64

Lesson 10.3 — Convolution, filters and systems 68

Lesson 10.4 — Optimization, conditioning and approximation budgets 71

Sources and provenance 75

Mathematics for Games and Computing

How to use this teaching volume

This is Package 03, the expanded teaching edition of Volume 02 in Computer Science and Computer Engineering for Games. It follows the programming bundle and retains the original module titles, lesson identifiers, learning outcomes, original assignment briefs and first three quiz questions. Explanations, worked derivations, lab steps, extra questions and original problem sets are additions for this edition. They do not silently replace the roadmap's external study requirements.

The course connects mathematics to engine work: requirements become quantified claims; positions and cameras become coordinate transformations; simulation becomes a differential-equation model; benchmark claims become statistical arguments; and texture/audio processing becomes sampling and filtering. Every formula is useful only under its assumptions. A smooth animation or a passing test is evidence, not a substitute for those assumptions.

Use the workbook alongside the handbook. Read the targeted R-code study assignment, work its relevant exercises and log the source/section/problem numbers. The 75 supplied problems provide a concrete starting bank with worked answers; they do not exhaust university-level practice in every named subject. R40 supports algebra repair, R41 expands multivariable calculus, and R42/R54 are deeper references rather than first-day requirements.

Program map

Module	Mathematical focus	Evidence produced
06	Logic, proof, counting and graphs	A mathematical specification notebook
07	Vectors, matrices, transforms and fitting	Tested geometry and a coordinate contract
08	Calculus, mechanics and numerical stepping	Analytic comparisons and timestep studies
09	Probability, inference and regression	A transparent small experiment
10	Gradients, sampling, filters and optimization	A numerical notebook assembled for Gate B

The original volume estimate is 500-800 study hours including exercises and projects. Gate B integrates the five module projects rather than adding five unrelated capstones. The work is self-study and does not confer a degree.

Reading code and numerical evidence

Code shown in each lesson is the body of its supplied runnable demonstration; the example file adds the import-path setup needed to run from this folder. Use `python run_example.py 06.1` or the corresponding ID. Expected output was captured from the tested environment; tiny formatting or floating-point differences may depend on interpreter/library behavior, so inspect differences rather than editing answers blindly.

Mathematical prose uses ^ for powers, while Python uses **. Core geometry uses column vectors, right-handed rotations, meters for example positions and a y-up world unless the local exercise says otherwise. A previous y-down terminal grid is a different coordinate convention, not an error to conceal. Code APIs and invalid-input policies are specified in the workbook. No claim of a production numerical library is made.

Keep three evidence categories separate: analytic results proved from assumptions; numerical/synthetic data generated from chosen models; and elapsed-time measurements on the recorded computer. Data filenames and metadata identify the category. The supplied benchmark is a Python list/set membership experiment, not a prediction of C++ engine or GPU performance.

Assessment and independent work

Each of the five lesson questions is scored 0-2: zero for absent/incorrect; one for a partly correct answer or a correct number without requested reasoning; two for a correct answer with the relevant argument. **Keep the original three-question threshold: questions 1-3 need at least 5/6. The full five-question quiz also needs at least 8/10.** Complete assigned study, the full practical assignment and the unaided transfer task before marking the lesson complete. Code-checker success alone is not completion.

The five module-review questions use the same two-point scale; suggested pass is 8/10. Correct missed work and solve a changed example. For the 15 written problems in each module, retain an independent attempt and a correction; proofs must identify assumptions and justified steps. At least two derivations/proofs per module must be reviewed, not merely checked for a final number.

The Gate B project uses the original 100-point rubric: correctness/requirements 30; tests/verification 25; reasoning/tradeoffs 15; quantitative error/performance analysis 10; reproducibility 10; communication/provenance 10. Suggested pass is 80/100, every essential check completed, and no unresolved critical defect in the claimed scope. A reviewer may accept another technically sound method when its assumptions and evidence are explicit.

Reference answers are study aids after an attempt. Explain all assisted code, record help received, and do the transfer task without following a supplied solution. Nothing here grants an academic credential. Passing a few numerical examples does not prove a mathematical theorem or demonstrate all of a semester's subject depth.

Module 06 — Discrete mathematical reasoning

Original prerequisites: Volume 01; complete the algebra diagnostic in START_HERE before proceeding.

Original module project: Write a mathematical notebook containing a truth-table specification, an induction proof, a graph model and a justified operation-count estimate. Include counterexamples to three false claims.

Expanded project: A mathematical specification notebook. A versioned document plus scripts containing five quantified world rules, complete door truth tables, two proofs, a six-node dependency graph, two recurrence analyses, and three counterexamples.

Acceptance: The proof reviewer can identify initialization/preservation/termination where applicable. Every graph edge has a stated meaning; cyclic input is rejected. The ten algebra-bridge items and all 15 module problems have independent attempts and corrections.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 06.1 — Algebra, functions, logarithms and trigonometry

Learning outcome: Translate a game relationship into an equation with a defined domain.

Start with a relationship, not a remembered formula

A useful mathematical model identifies quantities, their units, and the inputs for which its rules make sense. Suppose a unit moves a straight-line distance d at a constant positive speed v. Its travel time is t = d/v. This model deliberately omits obstacles, acceleration, pauses, and turning. A route planner may later supply a path length, but a straight-line distance is not automatically that path length.

An equation describes a relationship; an assignment performs an update. In mathematics, d = vt can be rearranged to v = d/t only when t is nonzero. A simulation assignment such as `distance = speed * time` does not make future changes to speed update the stored distance. Preserve the distinction learned in Volume 1.

Rearrange while preserving the domain

To solve 3t + 2 = 14, subtract 2 from both sides and then divide both sides by 3. Each operation preserves equality, yielding t = 4. Division by an expression is more delicate: cancelling x from x(x - 2) = 0 would discard the valid solution x = 0. Factoring instead gives x = 0 or x = 2. Squaring both sides can introduce extra solutions, so substitute candidates into the original equation.

For a real function sqrt(x - 4), the domain is x >= 4. For 1/(x - 4), it is x != 4. For log2(x - 4), it is x > 4. A domain is part of a function's contract. Rejecting impossible inputs clearly is more useful than allowing a calculation to produce a plausible-looking but meaningless result.

Exponents and logarithms describe scale

The identity 2^k = n is equivalent to k = log2(n) for n > 0. Repeatedly halving 1024 reaches 1 after ten divisions. For 1000, log2(1000) is not an integer. An algorithm that floors each half, and one that rounds each half up, can require different integer step counts. State the exact recurrence before turning a logarithm into a loop bound.

A logarithm needs a dimensionless argument. Writing log(seconds) without a reference scale is incomplete; log(t/t0) is meaningful when t and t0 use the same units. This habit becomes important when fitting curves or comparing quantities that use different measurement scales.

Angles and the unit circle

A radian measures arc length divided by circle radius. One revolution is 2*pi radians, so degrees convert by theta_rad = theta_deg*pi/180. In an x-right, y-up plane, the unit direction at angle theta measured counterclockwise from +x is (cos(theta), sin(theta)). A speed v then produces velocity (v*cos(theta), v*sin(theta)). Its displacement over dt is that velocity multiplied by dt.

The unit-circle identity cos(theta)^2 + sin(theta)^2 = 1 explains why the direction has length one. Multiplying its components by v scales the speed without changing its angle. Python's trigonometric functions use radians; displaying an angle in degrees does not change the function's input contract.

Exact mathematical zeros can appear as tiny floating-point values, which is a representation issue rather than a new geometric direction. [R04, R40]

Worked derivation

A unit travels 36 m at 6 m/s. The model requires d >= 0 and v > 0, so t = 36/6 = 6 s. At a heading of 90 degrees, theta = pi/2, giving velocity (0, 6) m/s in the stated y-up coordinates. During dt = 0.25 s, displacement is (0, 1.5) m.

Check the units: (m/s)*s = m. Check the magnitude: sqrt(0^2 + 6^2) = 6 m/s. Check the domain: speed zero does not produce a finite arrival time for a positive distance. It should be reported as an invalid input or unreachable state, not changed to an arbitrary small number.

For a rendering budget f = 120 frames/s, the interval is 1000/f = 8.333... ms/frame. This is an arithmetic target, not evidence that a machine can meet it.

Runnable demonstration

Run `python run_example.py 06.1` from the extracted package folder. Predict before executing.

```
import math
from mathkit import travel_time
print(f"travel time: {travel_time(36, 6):.1f} s")
angle = math.radians(90)
print(f"direction: ({math.cos(angle):.3f}, {math.sin(angle):.3f})")
print(f"halvings of 1024: {int(math.log2(1024))}")
```

Recorded expected output:

```
travel time: 6.0 s
direction: (0.000, 1.000)
halvings of 1024: 10
```

Practical assignment

Original brief: Solve distance = speed*time for each variable and state forbidden inputs. Convert 0, 90, 180 and 360 degrees to radians. Pass: solve ten algebra problems and explain logarithms using repeated halving before advancing.

1. Implement travel_time(distance, speed), requiring finite nonnegative distance and finite positive speed. Return seconds for inputs in meters and meters/second.
2. Complete the original ten algebra problems; the readiness bridge supplies concrete practice. Show all rearrangements and domain restrictions.
3. Tabulate directions at 0, 90, 180 and 360 degrees. Explain near-zero floating-point components rather than forcing all small numbers to zero.
4. Compare floor-halving and ceiling-halving for sizes 1, 2, 3, 1000 and 1024. Record the chosen recurrence.

Coding slice: Work in `labs/student/lab_06_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not pass the number 90 directly to a radian-based sine function, cancel a possibly zero factor, or round a logarithm before defining the discrete problem.

Independent transfer task

A unit moves at 8 m/s at 30 degrees for 0.5 s. Find its displacement in the stated y-up plane.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is log2(64)?
2. Convert 180 degrees to radians.
3. What restriction applies when computing speed = distance/time?
4. State the real domain of f(x) = sqrt(x-2)/(x-5).
5. Solve x(x-2)=0 without losing a solution.

Assigned study

R40: functions and domains, exponential/logarithmic functions, and the unit circle. R08: prerequisite review. R06: sums and asymptotic growth. Use the source topics, not guessed exercise numbers.

Original reading assignment retained: R08: prerequisite algebra review and trigonometric functions; R06: sums and asymptotics.

Lesson 06.2 — Logic, sets and quantified claims

Learning outcome: Express requirements precisely enough to test or prove.

Convert a requirement into a claim

A requirement such as "all living units have transforms" contains a domain and a predicate. Let U be the set of units and let live(u) and has_transform(u) be predicates. The precise claim is: for every u in U, live(u) implies has_transform(u). It does not say dead units lack transforms, or that transformed units must be living.

An implication P -> Q is equivalent to (not P) or Q. Its only false case is P true and Q false. When P is false, the implication places no obligation on Q. This is not a causal assertion. It is a condition that rules out one combination of truth values.

Quantifiers have different obligations

A universal claim requires every allowed case; an existential claim requires a witness. To disprove "every saved entity has a unique ID," one repeated ID is enough. To establish "some unit can reach the exit," provide one allowed unit and an actual route or a justified reachability argument.

Order matters. "Every player can choose some spawn point" allows different spawn points for different players. "There is a spawn point every player can choose" requires one common point. A two-player map with disjoint allowed spawn sets satisfies the first claim but not the second. A test generator must reflect the correct quantifier order.

Negation changes the quantifier

The negation of "for every u, P(u)" is "there exists u for which not P(u)." The negation of "there exists u for which P(u)" is "for every u, not P(u)." Negating only the predicate while keeping the quantifier is a frequent specification error.

For a universal statement over an empty set, there is no counterexample, so the statement is true. For an existential statement over that set, there is no witness, so it is false. Therefore a validator that checks every entity's ID can return true for a file with zero entities. A separate nonempty-world requirement is needed when emptiness is forbidden.

Sets, relations, and functions

A set contains distinct elements; a list can repeat elements and preserve an order. The set {2, 2, 5} has two elements, while the list [2, 2, 5] has three entries. A relation is a set of ordered pairs. A function is a relation assigning exactly one output to each input in its declared domain. A unit-to-transform mapping must therefore neither omit required units nor give a unit two incompatible authoritative transforms.

Conjunction means both conditions must hold; disjunction means at least one holds unless "exclusive or" is explicitly stated. For the door rule key OR (level >= 5 AND stamina >= 10), a key suffices regardless of the other two predicates. Parentheses communicate the intended grouping. Build the eight-row Boolean truth table first, then separately test the numeric comparisons at 4/5/6 and 9/10/11. The truth table does not replace boundary tests of how those predicates are computed. [R06]

Worked derivation

Suppose U = {A, B, C}, with health values 5, 0, and 8. The claim "every unit has positive health" is false because B supplies a counterexample. The claim "some unit has positive health" is true; either A or C is a witness.

For the guarded requirement live(u) -> has_transform(u), a dead B with no transform does not violate the requirement. A living A without a transform does. To require transforms for all units, remove the live guard rather than trying to repair the truth table.

Negate "every active unit has a valid owner": there exists an active unit whose owner is invalid. It is not "every active unit has an invalid owner."

Runnable demonstration

Run `python run_example.py 06.2` from the extracted package folder. Predict before executing.

```
from mathkit import implies
for p in (False, True):
    for q in (False, True):
        print(f"{int(p)} -> {int(q)} = {int(implies(p, q))}")
health = [5, 0, 8]
print("all positive:", all(h > 0 for h in health))
print("counterexample indices:", [i for i, h in enumerate(health) if h <= 0])
```

Recorded expected output:

```
0 -> 0 = 1
0 -> 1 = 1
1 -> 0 = 0
1 -> 1 = 1
all positive: False
counterexample indices: [1]
```

Practical assignment

Original brief: Formalize five requirements for the grid game using sets and quantifiers. Construct truth tables for a door-access rule. Pass: provide a counterexample or witness for each negated requirement.

1. Implement implies(p, q) for actual Boolean inputs, rejecting integer lookalikes under this exercise contract.
2. Write five quantified requirements for Training Yard or a small fictional unit world. State each domain explicitly.
3. Construct the eight-row key/level/stamina predicate table, then test the numeric boundaries independently.
4. For each requirement, provide a witness or counterexample for its negation. Include an empty collection and a repeated ID.

Coding slice: Work in `labs/student/lab_06_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Implication is not its converse. A validator passing a universal check on an empty collection does not establish that the collection is nonempty.

Independent transfer task

Two players have allowed spawn sets {A} and {B}. Which quantifier order is satisfied?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. When is P -> Q false?
2. Negate "every unit has positive health."
3. Why must the domain of a quantified statement be explicit?
4. What are the truth values of "all elements are positive" and "some element is positive" on an empty set?
5. Does P -> Q imply Q -> P? Give a game counterexample.

Assigned study

R06: propositions, predicates, sets, relations, and quantifier order. Complete a proof or truth-table exercise for each form used in the lab.

Original reading assignment retained: R06: propositions, predicates, sets and relations.

Lesson 06.3 — Proofs, induction and invariants

Learning outcome: Justify a program property instead of testing only examples.

A proof connects assumptions to a conclusion

Testing evaluates selected executions. A proof explains why a claim follows for every case within a mathematical model. Neither substitutes completely for the other: the proof may model integer arithmetic while the implementation uses finite-width numbers, or the implementation may not match the proven algorithm.

A direct proof starts with the premises and uses justified steps. To show an even integer plus an even integer is even, write them as 2a and 2b. Their sum is 2(a+b), which has the defining form of an even integer. A proof by contradiction assumes the negation of the desired result and derives an impossibility under the stated premises.

Induction is a controlled chain

For a claim P(n) over nonnegative integers, establish P(0). Then assume P(k) for an arbitrary allowed k and derive P(k+1). The inductive hypothesis is a temporary premise used to prove the step; it is not permission to assume the conclusion without connecting the algebra.

For S(n) = 1 + ... + n, the base S(0) = 0 matches 0(1)/2. Assuming S(k) = k(k+1)/2, add k+1 to obtain (k+1)(k+2)/2. This proves the formula at the next integer. Checking n=1, 2 and 3 alone does not establish the chain.

Put an invariant at a precise program boundary

To count living units, let i be the number already inspected and c the count of positive-health values among indices 0 through i-1. At the top of the loop, require 0 <= i <= N and c equals exactly that prefix count. Before the loop, i=0 and c=0, so the empty prefix establishes the invariant.

During an iteration, inspect the element at i. Increment c exactly when its health is positive, then increment i. The newly processed prefix has the correct count, preserving the invariant. At exit i=N, the prefix is the whole collection, so c is the desired total.

Termination is a separate obligation

The quantity N-i is a nonnegative integer that decreases by one per iteration. It cannot decrease forever while remaining nonnegative; this supplies a termination argument. An invariant by itself does not guarantee termination. A loop that never changes i might preserve a true statement and still run forever.

Separate partial correctness (the result is correct if the loop terminates) from total correctness (it also terminates under the premises). Record invalid-input behavior separately: if k exceeds the collection length, this exercise rejects the call instead of redefining the claimed prefix.

Connect the model back to code

The implementation reads finite health values and counts strictly greater than zero. A different rule, such as health >= 0, requires a different predicate and different boundary tests. Mathematical integers do not overflow; later native-code lessons will add range constraints. Code review should check initialization, the comparison, the update order, and the exit condition against the proof, not merely verify that the comments sound mathematical. [R06]

Worked derivation

Take health = [4, 0, -1, 9]. The prefix counts for k=0,1,2,3,4 are 0,1,1,1,2. At k=3, the invariant says c=1 because only the first entry among [4,0,-1] is positive. The next step reads 9, increments c to 2, and extends the prefix to four entries.

For the sum formula at n=100, n(n+1)/2 = 5050. The demonstration compares this with a loop as an implementation check. The induction proof, rather than those two matching numbers, justifies the general formula.

Runnable demonstration

Run `python run_example.py 06.3` from the extracted package folder. Predict before executing.

```
from mathkit import count_positive_prefix
health = [4, 0, -1, 9]
for k in range(len(health) + 1):
    print(f"prefix {k}: {count_positive_prefix(health, k)} living")
n = 100
print("sum and formula:", sum(range(n + 1)), n * (n + 1) // 2)
```

Recorded expected output:

```
prefix 0: 0 living
prefix 1: 1 living
prefix 2: 1 living
prefix 3: 1 living
prefix 4: 2 living
sum and formula: 5050 5050
```

Practical assignment

Original brief: Prove the formula 1+...+n = n(n+1)/2 by induction. Then write an invariant for counting living units in a prefix of a list. Pass: show initialization, preservation and termination explicitly.

1. Implement count_positive_prefix(values, k); allow an empty list with k=0, reject nonfinite data and invalid k, and do not modify inputs.
2. Write the full induction proof for 1+...+n, including n=0 and the algebraic step.
3. Write initialization, preservation, exit consequence, and a decreasing termination measure for the prefix-count loop.
4. Insert an off-by-one or >= defect in a disposable copy; find a counterexample and explain which proof-to-code correspondence fails.

Coding slice: Work in `labs/student/lab_06_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

A statement like "the count is correct" is too vague without saying which prefix, at what point in the loop, and under which health rule. Do not omit termination.

Independent transfer task

Change "positive health" to "health at least 3." Which parts of the proof change?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What are the two essential parts of induction?
2. What is 1+...+100?
3. Why does a mathematical proof not eliminate implementation tests?
4. Can a loop invariant be true while a loop never terminates?
5. State a precise invariant after processing i health values.

Assigned study

R06: induction, invariants, and state machines. Supply two independently written proofs in the module problem set; computed examples support but do not replace them.

Original reading assignment retained: R06: induction, invariants and proof methods.

Lesson 06.4 — Counting, graphs and recurrences

Learning outcome: Model combinations and recursive work without brute-force guessing.

Count choices without assuming independence of restrictions

If four equipment slots each allow three choices and choices impose no restrictions on one another, there are 3^4 = 81 loadouts. This is the product rule for combinatorial choices, not a probability assumption about how players select them. When a restriction couples slots, multiply only after conditioning on the relevant cases, or subtract a carefully counted forbidden set.

The sum rule adds counts of disjoint alternatives. Counting "red or heavy items" by adding all red and all heavy items double-counts items that are both. Inclusion-exclusion for two sets is |A union B| = |A| + |B| - |A intersection B|. Writing the overlap explicitly prevents an inflated estimate.

Model a graph before choosing its algorithm

A graph has vertices and edges. In a navigation graph, an edge might mean legal movement; in a build graph, it might mean one asset must precede another. Directed edges and undirected edges model different relationships. A simple undirected graph has no self-loop and at most one edge per unordered pair of vertices.

The count n(n-1)/2 follows because each of n vertices can pair with n-1 others, but that counts each undirected pair twice. Directed edges without self-loops have at most n(n-1) choices. These bounds are not interchangeable.

A startup order is a topological order

Let an edge A -> B mean A must finish before B begins. A topological order places every prerequisite before its dependent. A directed cycle prevents such an order: following its obligations eventually requires the first node to precede itself. Acyclic graphs can have multiple valid orders when some nodes are independent.

The reference routine repeatedly takes a zero-indegree node, outputs it, and removes its outgoing obligations. Remaining nodes with no zero-indegree choice reveal a cycle. It uses sorting to make tie-breaking reproducible, not because every topological-sort algorithm needs sorting. The supplied routine is an inspectable educational implementation; do not attribute the optimal O(V+E) bound to its repeatedly sorted Python lists.

A recurrence separates depth from total work

Define H(1)=0 and H(n)=1+H(floor(n/2)) for integer n>1. This counts floor-halvings and gives floor(log2(n)). With ceiling halves instead, the count is ceil(log2(n)). A recursive call chain's depth is not necessarily its total operation count.

For a single half-sized branch doing constant extra work, T(n)=T(n/2)+1 has logarithmic cost for powers of two. For two half-sized branches plus linear work, T(n)=2T(n/2)+n has n work at each of log2(n) levels, plus its leaves: the cost grows as n log n. The models have similar depth but very different totals.

Big-O states an upper-growth bound under a cost model. It is not a duration in milliseconds. A model may count comparisons while an actual program also spends time allocating, sorting, and accessing memory. Preserve both the symbolic argument and a measured experiment without treating either as the other. [R06]

Worked derivation

Use nodes assets, device, and scene, with assets -> scene and device -> scene. Either assets,device,scene or device,assets,scene is valid. Adding scene -> assets creates a cycle and eliminates every valid startup order.

For four three-choice slots, suppose the only forbidden situation is choosing option A in both slots 1 and 2. The forbidden count is 1*1*3*3 = 9, so 81-9 = 72 remain.

For floor-halving, 1000 becomes 500,250,125,62,31,15,7,3,1: nine divisions. For ceiling-halving it becomes 500,250,125,63,32,16,8,4,2,1: ten. Both must be distinguished from a real-valued logarithm.

Runnable demonstration

Run `python run_example.py 06.4` from the extracted package folder. Predict before executing.

```
from mathkit import halving_steps, topological_order
graph = {"assets": ["scene"], "device": ["scene"], "scene": []}
print("order:", topological_order(graph))
print("loadouts:", 3 ** 4)
print("complete graph edges:", 5 * 4 // 2)
print("halving steps:", halving_steps(1024))
```

Recorded expected output:

```
order: ['assets', 'device', 'scene']
loadouts: 81
complete graph edges: 10
halving steps: 10
```

Practical assignment

Original brief: Build a dependency graph for game startup and identify an invalid cycle. Count loadout combinations with and without a restriction. Pass: explain the recurrence for halving a search range and distinguish depth from total work.

1. Implement halving_steps(n) using repeated floor division, accepting positive integers only. Do not implement it by rounding a floating logarithm.
2. Draw a startup graph with at least six nodes, label edge meaning, give a valid order, then add an explicit invalid cycle.
3. Count the four-slot loadouts with and without the stated restriction and check by enumeration.
4. Derive both a one-branch halving recurrence and a two-branch divide-and-combine recurrence. Distinguish comparisons, depth, and elapsed time.

Coding slice: Work in `labs/student/lab_06_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not multiply counts across coupled choices, add overlapping alternatives without correction, or equate recursion depth with total work.

Independent transfer task

There are six nodes. Compare the maximum directed and undirected edge counts without self-loops.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. How many combinations arise from five independent binary choices?
2. How many edges can a simple undirected graph with five vertices have?
3. Why is a dependency cycle a problem for a simple startup order?
4. Four slots have three choices each; option A in both first slots is forbidden. How many loadouts remain?
5. Does the repeatedly sorted reference topological routine automatically have O(V+E) time?

Assigned study

R06: counting, directed graphs, recurrences, and asymptotic notation. Use a diagram or adjacency list with an explicit edge convention.

Original reading assignment retained: R06: counting, graphs, recurrences and asymptotic notation.

Module 07 — Linear algebra and game geometry

Original prerequisites: Module 06.1; basic Python functions.

Original module project: Build a tested vector/matrix library for a two-dimensional scene, then demonstrate a three-dimensional camera transformation with written coordinate conventions.

Expanded project: A tested geometry and fitting toolkit. A small vector/matrix implementation with tests, a parent-child scene calculation, a camera coordinate conversion, one quaternion/matrix agreement case, and a five-point least-squares report.

Acceptance: Include zero-vector rejection, noncommuting transforms, a singular system, known hand-derived transforms, q/-q agreement, and residual/conditioning distinctions. Record column-vector order, handedness, axes, and homogeneous point/direction rules.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 07.1 — Vectors, dot products and projections

Learning outcome: Use vectors for direction, distance and component measurements.

Separate locations from displacements

A point locates something in a coordinate system. Subtracting two points produces a displacement vector: target minus origin. Adding a displacement to a point produces another point. A vector's components depend on the chosen basis; the geometric displacement need not change when those coordinates change.

In a two-dimensional x-right, y-up plane, moving from (2,1) to (5,5) produces displacement (3,4). Its Euclidean length is sqrt(3^2+4^2)=5. That length uses the ordinary orthonormal Cartesian metric. If one axis has been rescaled without accounting for units, blindly summing component squares may no longer measure the intended physical distance.

A direction is a vector divided by its length

For nonzero v, its unit direction is u=v/||v||. Multiplying u by a speed creates a velocity with that speed. This avoids diagonal movement becoming faster merely because both input axes are active: (1,1) has length sqrt(2), not one.

The zero vector has length zero and no unique direction. Decide what it means in the application: stop moving, retain a prior orientation, or reject the direction request. The reference normalization function rejects it. A caller that wants a stationary fallback must handle that state explicitly rather than replacing it with an arbitrary direction deep inside the math library.

A direct sum of squares can overflow for very large components or underflow for very small ones. The reference first divides by the largest absolute component, normalizes that scaled vector, and therefore avoids squaring the original extremes. This is numerical design, not a change to the vector's intended direction.

The dot product measures a component

The dot product a·b is the sum of matching component products. It is a scalar. For nonzero vectors, a·b = ||a|| ||b|| cos(theta). A positive dot product means an acute angle; a negative one means an obtuse angle; zero means perpendicular directions when neither vector is zero.

For a unit direction u, the signed scalar component of v along u is v·u, and the vector projection is (v·u)u. For a nonunit direction b, the projection is ((v·b)/(b·b))b, provided b is nonzero. The omitted denominator is a common bug when an interface does not actually guarantee normalized input.

A front-facing test uses displacement d=target-position and facing f. The sign of f·d works without normalizing nonzero f. An angular field-of-view threshold needs lengths: f·d >= ||f|| ||d|| cos(theta_max). A target exactly at the observer has zero displacement and needs an explicit policy rather than a fabricated angle.

Orientation in three dimensions

In right-handed three-dimensional coordinates, the cross product a×b is perpendicular to both vectors, with components (ay*bz-az*by, az*bx-ax*bz, ax*by-ay*bx). Its length is the parallelogram area ||a|| ||b|| sin(theta). Swapping the operands negates the result. Parallel vectors produce zero, so they cannot determine a unique normal by normalization.

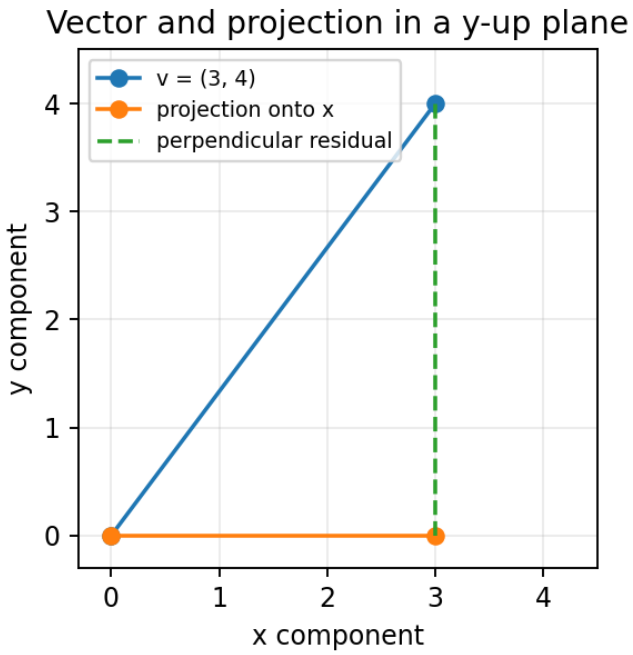
These operations answer different questions: dot gives alignment, cross gives an oriented perpendicular, and subtraction gives displacement. A good function name and type contract should make that distinction visible before the renderer or physics solver depends on it. [R07, R23]

Worked derivation

For v=(3,4), ||v||=5 and normalize(v)=(0.6,0.8). Its squared length is 0.36+0.64=1. Project v onto the x axis u=(1,0): v·u=3, so the projection vector is (3,0).

Project the same v onto b=(2,0), which is not unit length. The correct factor is (v·b)/(b·b)=6/4=1.5, giving 1.5*(2,0)=(3,0). Multiplying (v·b)b without the denominator would produce (12,0), four times the intended projection.

For facing (1,0), targets at displacements (2,1), (-2,1), and (0,4) are respectively in front, behind, and exactly to the side by the sign test.



Projection of (3,4) onto the x axis; the perpendicular residual is the y component.

Runnable demonstration

Run `python run_example.py 07.1` from the extracted package folder. Predict before executing.

```
from mathkit import dot, normalize
v = (3, 4)
u = normalize(v)
print("unit:", tuple(round(x, 3) for x in u))
print("x component:", dot(v, (1, 0)))
print("squared length:", dot(v, v))
try:
    normalize((0, 0))
except ValueError as error:
    print("rejected:", error)
```

Recorded expected output:

```
unit: (0.6, 0.8)
x component: 3.0
squared length: 25.0
rejected: zero vector has no unique direction
```

Practical assignment

Original brief: Implement length, dot and safe normalization. Use dot products to determine whether a target is in front of a character. Pass: test zero vectors, perpendicular vectors and opposite directions.

1. Implement dot(a,b) and normalize(v), documenting finite inputs, equal dimensions, and a deliberate zero-vector policy. Do not mutate the arguments.
2. Test perpendicular, opposite, and identical directions; include zero and extreme finite magnitudes in normalization.
3. Use target-position, not the target point alone, in a front-facing query. Decide what happens when target equals position.
4. Prove that the projection residual is perpendicular to a nonzero projection direction, then check a numerical case.

Coding slice: Work in `labs/student/lab_07_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Zero dot product does not establish two useful perpendicular directions when one operand is zero. Do not confuse a scalar component with a projection vector.

Independent transfer task

Find the projection of v=(2,2) onto b=(1,1), and then the rejection v-projection.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the length of (6,8)?
2. What is the dot product of (1,0) and (0,1)?

3. Why cannot the zero vector be normalized by the ordinary formula?
4. Why does unnormalized input (1,1) make diagonal movement faster than input (1,0) at the same scalar multiplier?
5. Compute the projection of (3,4) onto the nonunit vector (2,0).

Assigned study

R07: vectors, orthogonality, and projections. R23: vectors and coordinate systems. Complete both the analytic projection derivation and executable tests.

Original reading assignment retained: R07: vectors and projections; R23: Geometry and Transformations / Vectors.

Lesson 07.2 — Matrices, bases and linear systems

Learning outcome: Understand a matrix as a transformation rather than a table to memorize.

Read a matrix through its basis images

Using column vectors, a matrix's columns describe where the basis directions go. The matrix [[2,0],[0,1]] maps the x basis to (2,0) and the y basis to (0,1). Therefore (x,y) maps to (2x,y). This interpretation makes the multiplication rule a consequence of linearity rather than an arbitrary table procedure.

A linear transformation preserves addition and scalar multiplication. Translation alone is not linear in ordinary coordinates because it moves the zero vector; the next lesson adds homogeneous coordinates to represent translation in an augmented space. A basis contains independent vectors spanning the space. In two dimensions, two parallel nonzero vectors are not a basis: they cannot express displacements off their common line.

Composition has an order

For column vector p, applying A and then B gives B(Ap)=(BA)p. Read the rightmost factor first. The product entry (BA)[i,j] is a sum over matching inner indices. A matrix with r rows and k columns can multiply one with k rows and c columns; the result has r rows and c columns.

A rotation followed by a nonuniform scale usually differs from the reversed order. This is not a floating-point error: the exact transformations differ. Associativity, (AB)C=A(BC), permits regrouping in exact arithmetic, but does not permit swapping factors. Floating-point regrouping can also change the last few bits.

Solve equations without treating inverse as the default operation

For Ax=b, Gaussian elimination adds multiples of one equation to another to remove a variable. It transforms the system while preserving its solution set. A pivot is the coefficient used for elimination. Swapping rows to choose a larger available pivot reduces some avoidable numerical trouble, but cannot cure a fundamentally sensitive problem.

For 2x+y=5 and x-y=1, subtract half of the first equation from the second: -1.5y=-1.5. Thus y=1 and x=2. Verify both original equations, rather than trusting the intermediate arithmetic alone.

The reference solve2 routine scales the matrix, uses a row swap when useful, and rejects pivots that are small relative to its chosen scale. This is a stated numerical policy, not a proof that every rejected matrix is exactly singular. A different tolerance changes the supported problem envelope.

Rank and lost information

A singular square matrix lacks a unique inverse. A zero scale on one axis collapses distinct points together. More generally, a nonzero vector n with An=0 lies in the nullspace: x and x+n map to the same output. Rank counts independent output directions. A two-by-two matrix of rank one maps the plane into a line.

A singular system may have no solution or infinitely many solutions, depending on b. For equations x+y=2 and 2x+2y=4, every point on one line works. Replacing the second right side with 5 makes the equations inconsistent. "The solver failed" is therefore not a complete mathematical diagnosis.

If B contains basis vectors as columns, B*c converts coordinates c in that basis into the original basis. Recovering c means solving B*c=p. Document what each coordinate tuple means before combining transforms from different parts of an engine. [R07]

Worked derivation

Let R=[[0,-1],[1,0]] be a 90-degree counterclockwise rotation and S=[[2,0],[0,1]] a nonuniform scale. Start with p=(1,1).

Apply R first: Rp=(-1,1); then S gives SRp=(-2,1). Apply S first: Sp=(2,1); then R gives RSp=(-1,2). These exact outputs differ, demonstrating SR != RS.

For A=[[2,1],[1,-1]] and b=(5,1), elimination gives x=(2,1). The residual Ax-b is (0,0) in exact arithmetic. In numerical work, report a norm of that residual and the problem scale; a small residual alone will not later guarantee small solution error in an ill-conditioned problem.

Runnable demonstration

Run `python run_example.py 07.2` from the extracted package folder. Predict before executing.

```
from mathkit import matmul, solve2
rotation = ((0, -1), (1, 0))
scale = ((2, 0), (0, 1))
p = ((1,), (1,))
print("rotate then scale:", matmul(matmul(scale, rotation), p))
print("scale then rotate:", matmul(matmul(rotation, scale), p))
print("solve:", solve2(((2, 1), (1, -1)), (5, 1)))
```

Recorded expected output:

```
rotate then scale: ((-2.0,), (1.0,))
scale then rotate: ((-1.0,), (2.0,))
solve: (2.0, 1.0)
```

Practical assignment

Original brief: Implement 2-by-2 multiplication and solve two small linear systems by elimination on paper. Pass: show a pair of transforms that do not commute and explain a singular example geometrically.

1. Implement matmul(a,b) and solve2(a,b), with dimension checks and documented singular/small-pivot rejection. Test a row-swap case.
2. Show the rotation/scale noncommutativity example entirely on paper before checking it in code.
3. Solve two small linear systems by elimination and distinguish a unique, inconsistent, and underdetermined case.
4. Provide a nonzero nullspace vector for a singular matrix and verify that it explains two inputs with the same output.

Coding slice: Work in `labs/student/lab_07_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Row storage order in memory does not determine whether the mathematics uses row or column vectors. Keep representation, multiplication convention, and composition order separate.

Independent transfer task

Solve x+y=2 and 2x+2y=5. What fails?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Do matrices generally satisfy AB = BA?
2. What happens to invertibility when an axis is scaled to zero?
3. Why solve Ax=b without explicitly forming an inverse?
4. What are the output dimensions of a 3x2 matrix times a 2x4 matrix?
5. Give two different points that a zero x-scale maps to the same point.

Assigned study

R07: matrix multiplication, elimination, inverses, column space, nullspace, and change of basis. Work an elimination problem without forming an inverse.

Original reading assignment retained: R07: matrix multiplication, elimination, inverses and vector spaces.

Lesson 07.3 — Homogeneous transforms, cameras and quaternions

Learning outcome: Compose a scene using explicit coordinate conventions.

Write the coordinate contract first

This bundle uses column vectors, right-handed three-dimensional coordinates, x-right, y-up, and an active counterclockwise rotation about +z when viewed toward the origin from +z. A transform multiplies on the left. These are chosen conventions, not universal engine rules. The earlier terminal game's down-positive grid is a different coordinate system and needs an explicit conversion.

In homogeneous coordinates, represent a point as (x,y,z,1) and a direction as (dx,dy,dz,0). The affine transform with linear part A and translation t maps (p,1) to (Ap+t,1), and (d,0) to (Ad,0). The last row is (0,0,0,1). A translation must not change a pure direction.

Parent-child transforms are coordinate maps

Let M_parent convert parent-local coordinates into world coordinates, and M_child convert child-local coordinates into parent coordinates. Then M_world=M_parent*M_child. Each matrix has an input space and an output space. A chain is meaningful only when adjacent spaces match.

For a rigid pose, M=T(c)R means rotate local geometry by R, then translate by c. The inverse is R^T*T(-c), because an orthonormal rotation's inverse is its transpose. Reversing each operation without reversing their order gives the wrong inverse. Nonuniform scaling is not inverted by simply transposing the whole linear block.

The camera is another coordinate frame

A camera pose C maps camera-local coordinates into world space. The view matrix V=C^-1 maps world geometry into camera coordinates. A camera moving right makes fixed world geometry appear left in camera space; this is a change of viewpoint, not a modification to all world objects.

A perspective projection is a separate transformation. Its homogeneous output generally requires division by w, and clipping rules determine which points are representable or visible. This lesson's reference transform4 deliberately performs only the four-component multiply. It does not silently divide, clip, or impose a graphics API's depth range. Full perspective and clipping arrive in Volume 5.

Quaternions encode rotations, not positions

A unit quaternion in scalar-first form q=(w,x,y,z) represents an orientation. For a unit axis u and angle theta, q=(cos(theta/2), u*sin(theta/2)). The rotated vector can be obtained from q*(0,v)*q^-1, using quaternion multiplication. A quaternion has four components but represents a three-dimensional rotation with a unit-length constraint.

Both q and -q represent the same rotation. Consequently, interpolating components without handling that sign can take an unintended path or even produce the zero quaternion. A common normalized-linear interpolation first chooses the sign giving a nonnegative dot product, blends, and normalizes; it generally does not yield constant angular speed. Spherical interpolation addresses a different speed property and still needs consistent conventions.

Verification must challenge the convention

Test identity, a known quarter turn, translation of a point, translation of a direction, and a parent rotation that moves a child's offset. For an inverse round trip, compare within an absolute and relative tolerance tied to the numerical scale. A round trip by itself is weak: two implementations can share the same convention mistake and cancel it. Keep hand-derived expected positions as independent checks. [R07, R23]

Worked derivation

Take parent pose T(10,0,0)*Rz(pi/2) and child-local translation T(2,0,0). The child's origin first becomes (2,0,0) in parent coordinates, rotates to (0,2,0), and translates to (10,2,0) in world coordinates.

For T(10,20,30), point (1,0,0,1) becomes (11,20,30,1), but direction (1,0,0,0) stays unchanged. The difference follows directly from multiplication by the last component w.

For a z-axis quarter turn, q=(sqrt(2)/2,0,0,sqrt(2)/2). It rotates (1,0,0) to (0,1,0), agreeing with the matrix calculation. Test -q as well; the result should agree.

Runnable demonstration

Run `python run_example.py 07.3` from the extracted package folder. Predict before executing.

```
import math
from mathkit import translation, rotation_z, matmul, transform4, quat_rotate
parent = matmul(translation(10, 0, 0), rotation_z(math.pi / 2))
child = translation(2, 0, 0)
world = matmul(parent, child)
print("child origin:", tuple(round(x, 6) for x in transform4(world, (0, 0, 0, 1))))
print("direction:", transform4(translation(10, 20, 30), (1, 0, 0, 0)))
q = (math.cos(math.pi / 4), 0, 0, math.sin(math.pi / 4))
print("quaternion:", tuple(round(x, 6) for x in quat_rotate(q, (1, 0, 0))))
```

Recorded expected output:

```
child origin: (10.0, 2.0, 0.0, 1.0)
direction: (1.0, 0.0, 0.0, 0.0)
quaternion: (0.0, 1.0, 0.0)
```

Practical assignment

Original brief: Create a parented object and camera using documented conventions. Pass: test identity, inverse round trips within tolerance, a translated direction that remains unchanged, and a parent rotation that moves its child correctly.

1. Implement transform4(m,v) for a 4x4 matrix and a four-component column vector. It must not perform perspective division.
2. Create the parent-child chain above, a camera inverse round trip, and an independently derived quarter-turn test.
3. Verify points and directions have different translation behavior. Write an explicit grid-to-world conversion for y-down versus y-up.

4. Derive one axis-angle quaternion and compare its rotation with a matrix; check q and -q and reject the zero quaternion.

Coding slice: Work in `labs/student/lab_07_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not mix the terminal grid y-down convention with the geometry y-up convention, treat a direction as w=1, or confuse the camera pose with its inverse view transform.

Independent transfer task

A camera has no rotation and sits at (0,0,5). Where does world point (1,2,3) appear in camera coordinates?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Why commonly use w=0 for directions?
2. What does the view transform do?
3. Why document multiplication order even when using a library?
4. Why do q and -q require care during quaternion interpolation?
5. For a rigid pose C=T(c)R, what is the view transform?

Assigned study

R23: coordinate systems and transformations, points, vectors, and rotations. R07: orthonormal matrices and inverses. The quaternion material is a new Package 03 expansion; its conventions are explicitly specified here.

Original reading assignment retained: R23: Coordinate Systems and Transformations; R19: geometric transformations.

Lesson 07.4 — Eigenvalues, least squares and singular values

Learning outcome: Recognize stability and approximation problems in game data.

An eigenvector identifies a preserved direction

For a square matrix A, a nonzero vector v is an eigenvector when Av=lambda*v. Lambda is its eigenvalue. The transformed vector remains on the same line, though a negative eigenvalue reverses orientation. Zero is not an eigenvector because it would make every lambda trivially satisfy the equation.

For diag(2,0.5), the coordinate axes are eigenvector directions. Repeated application multiplies the x component by 2^n and the y component by 0.5^n. One grows while the other decays. Not every real matrix has real eigenvectors: a planar 90-degree rotation has no preserved real line. Complex eigenvalues later help describe oscillatory systems.

Eigenvalues help analyze repeated updates, but a statement about all long-term behavior needs assumptions about the matrix and its eigenstructure. In particular, unit-magnitude eigenvalues alone do not always guarantee bounded powers when a matrix has nontrivial Jordan structure. This bundle uses explicitly derived small examples rather than a blanket stability slogan.

Fit a line by defining the error you minimize

For observed pairs (xi,yi), ordinary least squares with intercept chooses slope a and intercept b to minimize S(a,b)=sum(yi-a*xi-b)^2. It assumes this squared vertical residual is the intended objective. It is not minimizing distance perpendicular to the line, absolute errors, or the largest error.

Set the partial derivatives to zero. The intercept equation makes the average residual zero, giving b=mean(y)-a*mean(x). Substitute into the slope equation to obtain a=Sxy/Sxx, where Sxy=sum((xi-mean(x))*(yi-mean(y))) and Sxx=sum((xi-mean(x))^2). A unique slope needs Sxx>0: the predictor must vary.

A good fit does not prove a physical law. Residuals should be examined for structure, outliers, and changing spread. If all measurements use one time value, additional precision in arithmetic cannot invent missing time variation.

Singular values measure stretching

For a real matrix, singular value decomposition writes A=U*Sigma*V^T, with orthonormal columns in the appropriate factors and nonnegative singular values. Geometrically, V^T rotates or reflects input coordinates, Sigma stretches along principal directions, and U maps those directions into the output space. Singular values are the square roots of eigenvalues of A^T A.

For diag(3,-2), eigenvalues are 3 and -2, while singular values are 3 and 2. Singular values describe stretch magnitudes, not signed orientation. A zero singular value signals a lost direction. For an invertible square matrix, the two-norm condition number is sigma_max/sigma_min.

Separate conditioning, algorithms, and coding mistakes

Conditioning asks how much the exact solution can change when the problem data changes. Algorithmic stability asks how much extra error the numerical method introduces. A coding error is an implementation defect, such as using the wrong column. They require different repairs.

Consider A=diag(1,0.000001). Its condition number is one million. A tiny change to the second component of b can produce a much larger absolute change in the second component of x. A small residual Ax-b therefore does not automatically imply a small forward error in x. Centering data and using an appropriate solver can reduce avoidable numerical error, but cannot eliminate missing information or the problem's intrinsic sensitivity. [R07]

Worked derivation

For x=(0,1,2,3,4) and y=(1.1,2.9,5.2,6.8,9.0), mean(x)=2 and mean(y)=5. Sxx=10 and Sxy=19.7, so slope=1.97 and intercept=1.06.

The residuals y-(1.97x+1.06) are (0.04,-0.13,0.20,-0.17,0.06); they sum to zero apart from floating-point rounding. The sum of squared residuals is 0.091. That is evidence about this chosen squared-error fit, not proof that future movement follows a straight line.

Apply diag(2,0.5) four times to (1,1): the result is (16,0.0625). The eigenvalues explain the growth and decay without executing all four updates.

Runnable demonstration

Run `python run_example.py 07.4` from the extracted package folder. Predict before executing.

```
from mathkit import fit_line
xs = (0, 1, 2, 3, 4)
ys = (1.1, 2.9, 5.2, 6.8, 9.0)
slope, intercept = fit_line(xs, ys)
print(f"line: y = {slope:.3f} x + {intercept:.3f}")
print("residuals:", [round(y - (slope*x + intercept), 3) for x, y in zip(xs, ys)])
x, y = 1.0, 1.0
for _ in range(4):
    x, y = 2*x, 0.5*y
print("four diagonal updates:", (x, y))
```

Recorded expected output:

```
line: y = 1.970 x + 1.060
residuals: [0.04, -0.13, 0.2, -0.17, 0.06]
four diagonal updates: (16.0, 0.0625)
```

Practical assignment

Original brief: Fit a line to five noisy position/time observations and report residuals. Analyze repeated multiplication by a diagonal matrix. Pass: distinguish data noise, poor conditioning and a programming error in your report.

1. Implement fit_line(xs,ys), returning slope then intercept, and reject unequal lengths, fewer than two pairs, nonfinite data, and a constant predictor.
2. Fit the five supplied measurements and report every residual, their sum, and the squared-error objective.

3. Analyze powers of two diagonal matrices symbolically and compare with code; include a negative eigenvalue.
4. Construct a diagonal sensitivity example and distinguish noise, conditioning, and a deliberately introduced indexing defect.

Coding slice: Work in `labs/student/lab_07_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Singular values are not generally signed eigenvalues. A tiny residual, visually good fit, or stable implementation cannot by itself remove an ill-conditioned problem.

Independent transfer task

For diag(4,0.25), find the singular values, two-norm condition number, and two updates of (1,1).

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What are the eigenvalues of diag(3,2)?
2. What objective does ordinary least squares minimize?
3. Can a stable algorithm make an ill-conditioned problem insensitive?
4. Can a 90-degree planar rotation have a nonzero real eigenvector?
5. Why does fitting a slope fail when every xi is equal?

Assigned study

R07: eigenvalues, diagonalization, least squares, and singular value decomposition. Derive the centered line-fit equations before using the reference routine.

Original reading assignment retained: R07: eigenvalues, least squares and singular value decomposition.

Module 08 — Calculus, mechanics and numerical simulation

Original prerequisites: Modules 06.1 and 07.1; work through the linked calculus problem sets.

Original module project: Build a projectile and spring simulator with analytic comparison cases, timestep refinement, unit checks and a report separating model error from numerical error.

Expanded project: Projectile and spring laboratory. Analytic equations, tested particle/spring functions, timestamped trajectory CSVs, and a fixed-horizon refinement report comparing explicit and semi-implicit Euler.

Acceptance: Include h=0.1,0.05,0.025 at T=2 seconds for the m=1,k=100 undamped spring, plus a bounded unstable h=0.25 case. State maximum and final position errors, energy behavior, units, and omitted physics.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 08.1 — Limits, derivatives and rates

Learning outcome: Connect derivatives to velocity, acceleration and local approximation.

A derivative is a limit of average rates

For a position function x(t), the average velocity over a nonzero interval h is (x(t+h)-x(t))/h. The instantaneous velocity is the limit of that quotient as h approaches zero, when the limit exists. It describes the local behavior of a mathematical function, not an instruction to divide two identical floating-point samples by zero.

For x(t)=t^2, expand (t+h)^2-t^2=2th+h^2. Dividing by h gives 2t+h. Taking the limit gives x'(t)=2t. At t=3 the exact derivative is 6; the forward difference with h=0.1 is 6.1. The difference is a predictable approximation error, not a bug.

A function can be continuous without being differentiable. The absolute-value function |t| has left slope -1 and right slope +1 at zero, so it has no ordinary derivative there. A collision, clamp, or branch in a game model can create similarly nonsmooth behavior. Check the mathematical model before interpreting a numerical derivative.

Carry units through the derivative

Position measured in meters differentiated with respect to seconds yields meters/second. Differentiating velocity yields acceleration in meters/second^2. If an animation curve is parameterized by normalized progress u rather than time t, its derivative with respect to u is not automatically a physical velocity. The chain rule supplies dx/dt=(dx/du)*(du/dt).

Basic rules allow compositional reasoning

The derivative of t^n is n*t^(n-1) for the usual polynomial cases. Sums differentiate term by term. For a product fg, the derivative is f'g+fg'; it is not f'g'. For a composition f(g(t)), it is f'(g(t))*g'(t).

For x(t)=(2t+1)^3, the inner derivative is 2 and the outer derivative is 3(2t+1)^2, giving 6(2t+1)^2. Omitting the factor 2 treats the internal time scaling as though it had no effect. This same error appears in transformed geometry and parameterized animation.

A tangent approximation x(t+h) approximately equals x(t)+x'(t)h for small h. It is local. A large step through a sharply curved trajectory can make an accurate derivative produce a poor position prediction.

Numerical differences balance two errors

The centered estimate is (f(x+h)-f(x-h))/(2h). For sufficiently smooth functions, its leading truncation error is proportional to h^2, whereas the forward estimate commonly has error proportional to h. These orders describe a limiting regime with bounded higher derivatives, not every possible step size or function.

As h becomes tiny, subtraction of nearly equal floating-point numbers loses useful relative precision, and x+h may even round back to x. The reference rejects an unresolvable h. No single absolute perturbation is suitable for every coordinate scale. Use an error table over several h values, compare

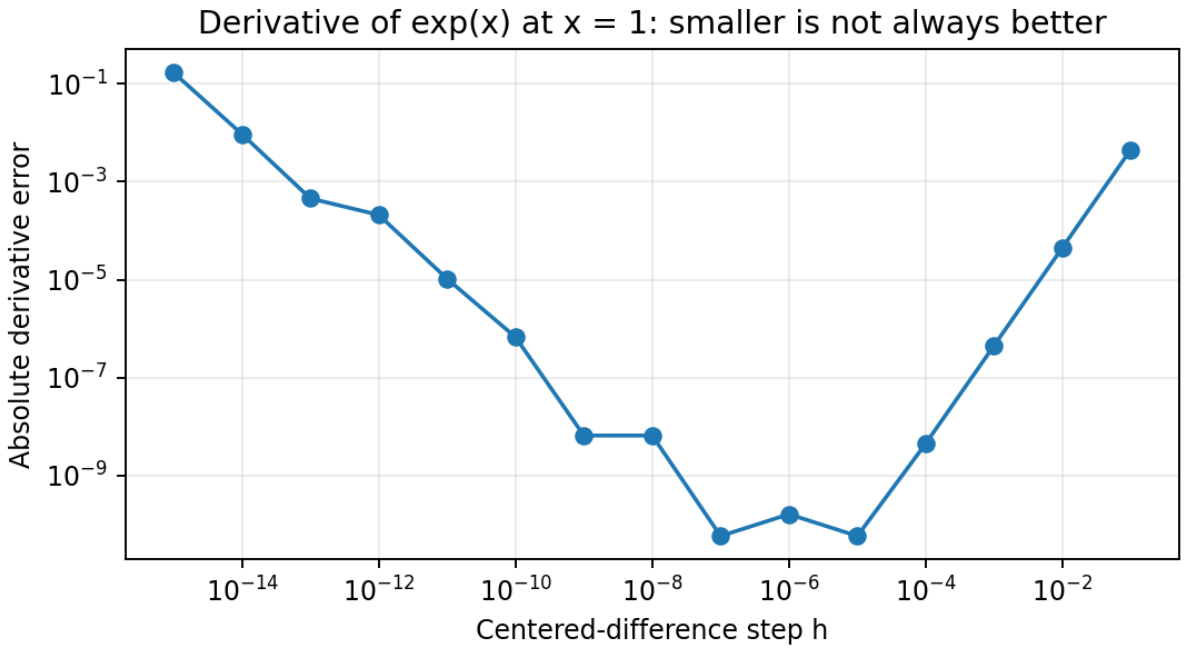
to an analytic derivative, and explain the observed region of improvement or deterioration. [R08, R04]

Worked derivation

Let f(t)=t^3 and evaluate at t=2. The exact derivative is 3t^2=12. Expanding the centered numerator gives (t+h)^3-(t-h)^3=6t^2*h+2h^3. After dividing by 2h, the estimate is 3t^2+h^2.

At h=0.1, 0.01 and 0.001, the exact-arithmetic estimates are 12.01, 12.0001 and 12.000001. Reducing h by ten reduces this truncation error by one hundred. Floating-point results follow that pattern only while rounding is small enough.

For x(t)=3t^2 meters with t in seconds, v(t)=6t m/s and a(t)=6 m/s^2. At t=2 s, the velocity is 12 m/s.



Centered differences for exp at x=1: reducing h eventually encounters floating-point limits. Unresolvable steps are excluded from the plot but recorded in the data.

Runnable demonstration

Run `python run_example.py 08.1` from the extracted package folder. Predict before executing.

```
from mathkit import centered_difference
f = lambda t: t ** 3
x = 2.0
exact = 3 * x * x
for h in (0.1, 0.01, 0.001):
    estimate = centered_difference(f, x, h)
    print(f"h={h:.3f} estimate={estimate:.6f} error={abs(estimate-exact):.6f}")
```

Recorded expected output:

```
h=0.100 estimate=12.010000 error=0.010000
h=0.010 estimate=12.000100 error=0.000100
h=0.001 estimate=12.000001 error=0.000001
```

Practical assignment

Original brief: Compare analytic and finite-difference derivatives of three functions over several step sizes. Pass: include units, an error table and an explanation of why the smallest tested h need not be best.

1. Implement centered_difference(f,x,h), requiring finite inputs, positive resolvable h, and finite function evaluations.
2. Compare analytic and numerical derivatives for t^2, t^3, and sin(t), using at least five step sizes. State radians for the sine argument.
3. Include the corner |t| at zero and explain why a centered value of zero does not establish differentiability.
4. Record absolute errors, units, and the approximation/rounding tradeoff. Do not smooth away a failed case without documenting it.

Coding slice: Work in `labs/student/lab_08_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

The smallest perturbation is not automatically the most accurate. A numerical slope at a corner is not proof that an ordinary derivative exists.

Independent transfer task

For x(t)=(3t-1)^2, derive velocity and acceleration, then evaluate at t=1.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Differentiate x(t)=3*t*t.
2. What units does acceleration have when position is in meters?
3. Why can excessively small finite-difference steps be inaccurate?
4. Differentiate (2t+1)^3.
5. Why is |t| not differentiable at t=0?

Assigned study

R08: derivative definition, basic rules, product and chain rules, and tangent approximation. R04: finite floating-point values and mathematical functions.

Original reading assignment retained: R08: differentiation, product rule and chain rule.

Lesson 08.2 — Integrals, accumulation and continuous probability

Learning outcome: Interpret integration as accumulated contribution.

Accumulation adds small contributions

An integral adds contributions across an interval. If v(t) is velocity, a tiny interval dt contributes approximately v(t)dt to displacement. The definite integral from a to b combines the limiting sum of such contributions. For a continuous integrand, this can be formalized as a limit of Riemann sums.

The integral is signed. A character moving forward and then backward can return to the starting position with zero net displacement while traveling a positive total distance. Integrating v gives displacement; integrating |v| gives distance in one-dimensional motion. The two are different objectives.

Derivatives and integrals are connected

If F is an antiderivative of f on the interval under the relevant regularity assumptions, the fundamental theorem of calculus gives the definite integral as F(b)-F(a). For f(t)=2t, use F(t)=t^2. The integral from 0 to 3 is 9, not the final velocity 6 multiplied by three seconds.

An indefinite integral represents a family of antiderivatives differing by a constant. Initial conditions fix that constant in a motion model. Integrating acceleration a(t) gives a velocity change; the initial velocity must still be added. Integrating velocity gives a position change; the initial position remains necessary.

Quadrature is a specified discrete approximation

Divide [a,b] into n equal panels of width h=(b-a)/n. The left rectangle rule samples f(a+i*h) for i=0,...,n-1 and multiplies their sum by h. The composite trapezoid rule gives half weight to the endpoints and full weight to internal samples.

For increasing f, left rectangles underestimate on a forward interval. For a curved function, trapezoids may overestimate or underestimate depending on curvature. Trapezoids integrate linear functions exactly in exact arithmetic; they do not integrate every smooth function exactly. Increasing n is useful only while model errors and numerical rounding are understood.

The reference allows a>b and returns the signed result using a negative panel width. Zero-width intervals return zero. It rejects nonpositive panel counts and unresolvable widths. Those are interface decisions that tests must match rather than infer from a library name.

Continuous probability uses areas, not point masses

A probability density p(x) is nonnegative and integrates to one over its support. Probability over an interval is the integral of the density there. For a continuous distribution, the probability at an exact single point can be zero even where the density is positive. Density values may exceed one because they are not themselves interval probabilities.

For a uniform position in [0,0.5] meters, the density is 2 per meter. The probability of [0.1,0.2] meters is 2*0.1=0.2. Units cancel. This same distinction later matters when sampling directions for rendering: a density must be specified with respect to a measure.

Check refinement against an independent answer

Use functions with known antiderivatives before trusting a complex simulation. At fixed total interval, compare n, 2n, and 4n panels. Report error against the analytic integral, not merely agreement between two versions of the same routine. A discontinuity or singularity requires a deliberate policy; blindly adding samples is not an explanation. [R08, R09]

Worked derivation

For f(t)=t^2 on [0,1], the exact integral is 1/3. With four panels, h=1/4. The left sum is h*(0^2+(1/4)^2+(1/2)^2+(3/4)^2)=0.21875.

The trapezoid sum is h*(0/2+(1/4)^2+(1/2)^2+(3/4)^2+1/2)=0.34375. Its error is 0.0104167. With eight panels, the result is 0.3359375 and the error is 0.00260417, one quarter as large. This is consistent with second-order composite trapezoid behavior for this smooth function.

For v(t)=2t on [0,3], the trapezoid rule is exact in exact arithmetic for any positive panel count, yielding displacement 9.

Runnable demonstration

Run `python run_example.py 08.2` from the extracted package folder. Predict before executing.

```
from mathkit import integrate
for n in (4, 8, 16):
    left = integrate(lambda t: t*t, 0, 1, n, "left")
    trap = integrate(lambda t: t*t, 0, 1, n, "trapezoid")
    print(f"n={n:2} left={left:.6f} trapezoid={trap:.6f}")
print("exact:", round(1/3, 6))
print("linear trapezoid:", integrate(lambda t: 2*t, 0, 3, 4))
```

Recorded expected output:

```
n= 4 left=0.218750 trapezoid=0.343750
n= 8 left=0.273438 trapezoid=0.335938
n=16 left=0.302734 trapezoid=0.333984
exact: 0.333333
linear trapezoid: 9.0
```

Practical assignment

Original brief: Implement rectangle and trapezoid integration for v(t)=2*t and a curved function. Pass: compare to an analytic integral, halve the step and report how error changes.

1. Implement integrate(f,a,b,n,method), supporting left and trapezoid rules with the documented signed-interval contract.
2. Compare t^2, 2t, and a chosen curved function against antiderivatives using n,2n,4n panels.
3. Include reversed limits, an equal-endpoint interval, invalid n, and a function returning a nonfinite value.

4. Write a displacement-versus-distance example and a normalized density calculation with units.

Coding slice: Work in `labs/student/lab_08_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

A density is not a probability at a point. Net displacement is not total distance, and multiplying final velocity by total time is not a general integration rule.

Independent transfer task

Velocity is +2 m/s for one second and -2 m/s for the next second. Find displacement and distance.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Integrate constant velocity 5 over two seconds.
2. What is the integral of 2*t from 0 to 3?
3. Why can an integral be zero despite nonzero motion?
4. Can a probability density equal 2 without violating probability bounds?
5. What is the integral of t^2 from 1 to 0?

Assigned study

R08: definite integrals, both fundamental-theorem connections, probability applications, and numerical integration. R09: continuous distributions when available after the probability module begins.

Original reading assignment retained: R08: integration and the fundamental theorem; R23: Monte Carlo Integration introduction.

Lesson 08.3 — Newtonian mechanics and differential equations

Learning outcome: Model motion from forces with clear simplifying assumptions.

Specify the physical state and assumptions

For a one-dimensional particle, use state (x,v), where x is position and v is velocity. The differential equations are dx/dt=v and dv/dt=F_net/m for constant positive mass m. Initial position and velocity select a trajectory; forces alone do not tell you where the particle starts.

A model is useful because it omits details deliberately. This bundle's projectile model ignores drag, collision, rotation, and variation in gravity. The spring model is one-dimensional, linear, and attached to a fixed rest position. These are appropriate learning models, not claims that every game object or physical mechanism obeys them exactly.

Force, acceleration, and impulse have different units

Force measured in newtons equals kg*m/s^2. Dividing by mass gives acceleration. Momentum is m*v, measured in kg*m/s. Impulse is the integral of force over time, measured in N*s, and changes momentum.

An instantaneous gameplay knockback may be modeled as an impulse, producing delta_v=J/m. Applying the same force for a duration produces an impulse only after accounting for that duration. Adding a force directly to velocity mixes incompatible units and makes behavior accidentally depend on the update frequency.

Constant acceleration supplies an analytic reference

When a is constant, integrate dv/dt=a to get v(t)=v0+a*t. Integrating that velocity gives x(t)=x0+v0*t+0.5*a*t^2. These formulas are exact within the model in real arithmetic. They are useful independent reference cases for a numerical integrator.

Use a signed coordinate convention. With y-up, gravity can be a=-g. A negative velocity indicates downward motion; it is not inherently invalid. The exercise restricts elapsed time to t>=0 because it models forward simulation, even though the algebraic formula can be evaluated at negative t.

Springs and damping depend on state

For displacement x from equilibrium, a linear spring exerts -k*x. The stiffness k has units N/m. Linear viscous damping exerts -c*v, where c has units N*s/m. Thus acceleration is (-k*x-c*v)/m. Positive k restores toward equilibrium; nonnegative c opposes velocity.

For an undamped spring, omega=sqrt(k/m) has units radians/second, and the continuous period is 2*pi/omega when k>0. With initial state x0 and v0, the analytic undamped solution is x(t)=x0*cos(omega*t)+(v0/omega)*sin(omega*t). Its velocity follows by differentiation. The k=0 case needs a separate constant-velocity model rather than division by zero.

Energy checks test the model as well as the implementation

Mechanical energy is E=0.5*m*v^2+0.5*k*x^2. Differentiating along the continuous spring equations gives dE/dt=-c*v^2. With c=0 the ideal model conserves energy; with c>=0 it does not gain energy from damping. A numerical update can violate that property when the timestep or method is unsuitable.

A visually smooth curve is therefore weak evidence. Record equations, units, initial conditions, omitted effects, and an analytic comparison. Distinguish a model limitation from an integration error or an implementation bug. [R08, R28; derivations in this lesson]

Worked derivation

Take x0=0 m, v0=4 m/s, a=-10 m/s^2, and t=0.5 s. The analytic position is 0+4*0.5+0.5*(-10)*0.5^2=0.75 m; velocity is 4-10*0.5=-1 m/s. The object is above its start but already moving downward.

For m=2 kg, k=50 N/m, c=1 N*s/m, x=0.2 m, and v=0, the force is -10 N and acceleration -5 m/s^2. A semi-implicit step of 0.01 s gives v_new=-0.05 m/s and x_new=0.1995 m. That is a numerical step, not the exact damped-spring solution.

Runnable demonstration

Run `python run_example.py 08.3` from the extracted package folder. Predict before executing.

```
from mathkit import constant_acceleration, spring_step
x, v = constant_acceleration(0, 4, -10, 0.5)
print(f"projectile: x={x:.3f} m, v={v:.3f} m/s")
x, v = spring_step(0.2, 0, 2, 50, 1, 0.01, "semi")
print(f"spring: x={x:.5f} m, v={v:.5f} m/s")
```

Recorded expected output:

```
projectile: x=0.750 m, v=-1.000 m/s
spring: x=0.19950 m, v=-0.05000 m/s
```

Practical assignment

Original brief: Implement a particle under constant acceleration and a damped spring. Pass: compare the constant-acceleration trajectory to its analytic solution and state what collision, rotation and air-resistance effects are omitted.

1. Implement constant_acceleration(x0,v0,a,t), returning position and velocity with finite values and t>=0.
2. Use the spring model to run both damped and undamped examples. Record m,k,c,initial state, units, and omitted effects.
3. Derive the constant-acceleration solution and the continuous spring energy-rate identity on paper.
4. Compare the numerical undamped spring against its analytic sine/cosine trajectory at identical timestamps.

Coding slice: Work in `labs/student/lab_08_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Force is not velocity, mass is not weight, and energy conservation belongs first to a declared continuous model. Numerical damping or growth must be measured rather than hidden.

Independent transfer task

A 2 kg body receives a 3 N*s impulse. What velocity change results? Compare with 3 N applied for 0.5 s.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What acceleration results from 10 N on a 2 kg mass?
2. How does impulse differ from force?
3. Why must initial conditions accompany a differential equation?
4. Write the state equations for a linear damped spring about x=0.
5. What is dE/dt for the ideal linear damped spring?

Assigned study

R08: antiderivatives and differential-equation applications. R28 is a later simulation API comparison, not a substitute for deriving these particle equations. No Box2D installation is required here.

Original reading assignment retained: R28: simulation, bodies, forces and impulses; R08: differential-equation applications.

Lesson 08.4 — Time integration, stability and error

Learning outcome: Choose and test a numerical update rather than assuming smaller code is correct.

Update order changes the method

Explicit Euler evaluates all derivatives at the old state: x_new=x+v*h and v_new=v+a(x,v)*h. Semi-implicit Euler first updates velocity, then uses v_new in the position update. This is a different numerical method, not a stylistic rearrangement of equivalent assignments.

For constant acceleration, the exact position increment is v*h+0.5*a*h^2. Explicit Euler omits the second term; semi-implicit Euler adds a*h^2. One undershoots and the other overshoots according to the sign of a. Both are first-order methods for position over a fixed total duration in this setting.

Accuracy and stability ask different questions

Accuracy concerns how close the computed solution is to the intended reference. Stability concerns whether errors or modes grow uncontrollably under the discretized update. A method can be stable but inaccurate at a coarse step; a small local error can still accumulate into unacceptable long-term drift.

Fixed timesteps simplify comparison and replay, but they do not guarantee stability or bit-identical results across systems. A timestep needs units, a total horizon, and a relationship to the system's fastest relevant motion.

Derive a concrete instability instead of guessing

For an undamped spring, acceleration=-omega^2*x. Explicit Euler gives x_new=x+h*v and v_new=v-h*omega^2*x. Substitute these into E=0.5*m*v^2+0.5*k*x^2, using k=m*omega^2. The cross terms cancel and E_new=(1+omega^2*h^2)*E.

For any h>0 and nonzero energy, the factor is greater than one. Repeated explicit steps therefore grow the oscillator's energy, even though the continuous model conserves it. At a fixed physical horizon, refining h reduces this drift; at a fixed nonzero h over arbitrarily long time, it persists.

For semi-implicit Euler, the oscillator update matrix has determinant 1 and trace 2-(omega*h)^2. For 0<omega*h<2, its eigenvalues are a distinct complex-conjugate pair of unit magnitude, yielding bounded motion for this particular linear system. At the boundary omega*h=2, boundedness is not generally guaranteed; beyond it there is a growing mode. Do not transfer this exact condition unchanged to nonlinear or damped systems.

Measure error at matching physical times

Comparing 100 steps at h=0.01 with 100 steps at h=0.005 compares one second with half a second. For a refinement study, keep T fixed and use N=T/h steps, choosing compatible h values. Include the initial state and compute timestamps as i*h rather than accumulating an unrelated wall-clock timer.

Record maximum position error, final position error, energy ratios, and nonfinite or divergence status. A deliberately unstable case belongs in the report. Do not delete it merely because it spoils the plot.

Separate the error sources

Model error comes from omitted or simplified physics. Discretization error comes from replacing the continuous evolution by a numerical update. Roundoff error comes from finite arithmetic. Measurement error belongs to observations used to fit or evaluate the model. More integration steps may reduce one source without changing the others.

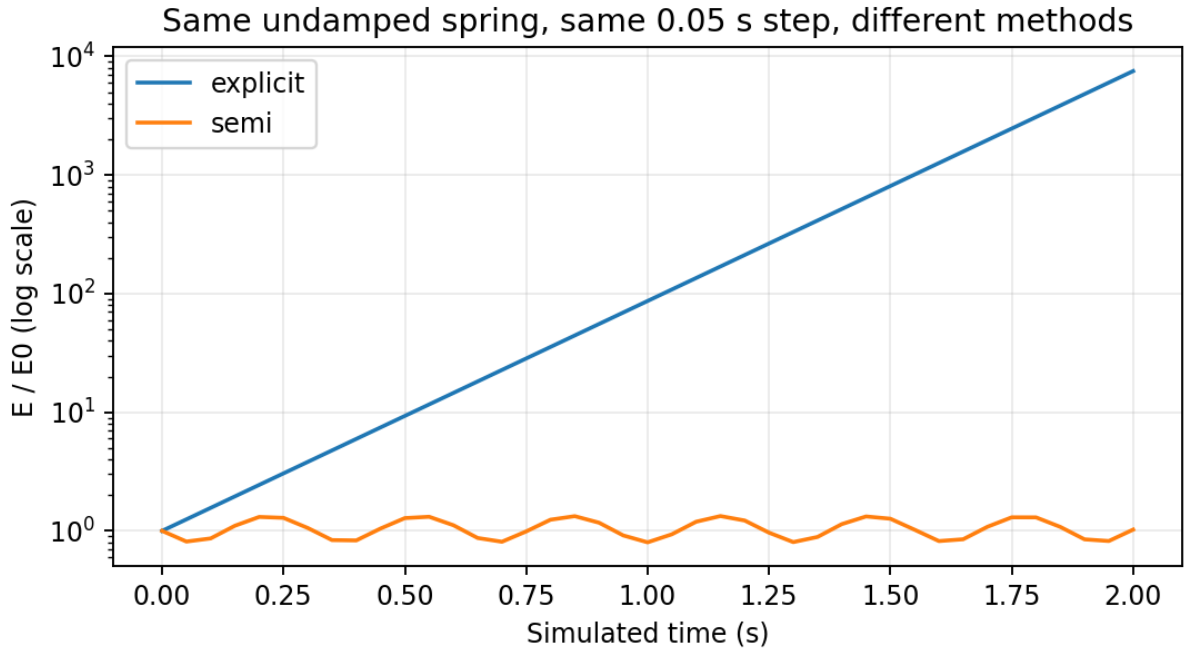
For larger projects, higher-order and implicit methods offer different cost/stability tradeoffs. This lesson does not claim that semi-implicit Euler is universally superior; it establishes a repeatable way to compare methods on a stated problem. [R08, R07; oscillator derivations in this lesson]

Worked derivation

Choose m=1, k=100, c=0, x0=1, v0=0. Then omega=10 and E0=50. With h=0.05, omega*h=0.5. Explicit Euler multiplies energy by 1.25 each step, so after 20 steps E/E0=1.25^20, approximately 86.736.

The semi-implicit run at the same h and one-second horizon has a bounded but oscillating energy; the supplied run ends near E/E0=0.800291. Ending below the initial energy does not prove exact energy conservation or monotonic damping.

For h=0.25, omega*h=2.5 exceeds the linear semi-implicit stability bound. Include this adverse case over a bounded horizon and report its growth rather than extrapolating indefinitely into overflow.



One chosen undamped oscillator, identical timestep and final time. Explicit Euler grows energy; semi-implicit Euler oscillates in energy and is not exact.

Runnable demonstration

Run `python run_example.py 08.4` from the extracted package folder. Predict before executing.

```
from mathkit import spring_step
for method in ("explicit", "semi"):
    x, v = 1.0, 0.0
    for _ in range(20):
        x, v = spring_step(x, v, 1, 100, 0, 0.05, method)
    energy = 0.5*v*v + 50*x*x
    print(f"{method}: E/E0={energy/50:.6f}")
print(f"explicit prediction: {(1 + 100*0.05**2)**20:.6f}")
```

Recorded expected output:

```
explicit: E/E0=86.736174
semi: E/E0=0.800291
explicit prediction: 86.736174
```

Practical assignment

Original brief: Compare explicit and semi-implicit Euler on an undamped spring at three timesteps. Pass: plot or tabulate position and energy error, identify an unstable case, and avoid calling visual smoothness proof of physical accuracy.

1. Implement spring_step(x,v,mass,stiffness,damping,dt,method), supporting explicit and semi updates and rejecting invalid physical parameters.
2. Compare both methods at h=0.1,0.05,0.025 over the same two-second horizon, against the analytic undamped solution.
3. Add an unstable semi case h=0.25 for m=1,k=100 and a bounded run length. Preserve energy and error tables, not only an animation.
4. Derive the explicit energy-growth identity; explain at least one model error that timestep refinement cannot repair.

Coding slice: Work in `labs/student/lab_08_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Equal step counts are not equal simulation durations. Smoothness is not accuracy, and a fixed step does not make an unstable update stable.

Independent transfer task

For m=1,k=25 and an undamped spring, what open interval of positive h satisfies the derived semi-implicit condition?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Which velocity does semi-implicit Euler use for the position update?
2. Does a fixed timestep guarantee a stable simulation?
3. What is a useful check beyond watching the animation?

4. For explicit Euler on an undamped spring, what is the one-step energy factor?
5. Why must a refinement study hold the final physical time fixed?

Assigned study

R08: differential equations and local approximations. R07: eigenvalues for the small update matrix. The original roadmap cross-references animation through R19/R23; use the explicit oscillator derivation here rather than assuming R23 contains a general simulation course.

Original reading assignment retained: R23: physically based animation topics via R19; R28: simulation stepping guidance.

Module 09 — Probability, statistics and evidence

Original prerequisites: Modules 06 and 08.2.

Original module project: Create a reproducible benchmark experiment and a loot-distribution simulation. Report assumptions, uncertainty, representative workloads and one result that did not support the original hypothesis.

Expanded project: Randomness and performance evidence. A detector contingency table, reward-distribution experiment with recorded seeds, and one locally measured comparison with randomized paired ordering, raw results, descriptive summaries, qualified uncertainty, and a held-out prediction exercise.

Acceptance: Synthetic and measured datasets remain separately labeled. Both algorithms produce equivalent answers. Report workload, machine/software boundary, warmup, trials, inclusion rules, percentile convention, and independence limitations. A null or adverse result is acceptable.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 09.1 — Probability and conditional reasoning

Learning outcome: Model uncertainty without confusing independent and conditional events.

Define the sample space and event

A probability model assigns nonnegative weights summing to one over possible outcomes. An event is a set of outcomes. For two independent fair coin flips, the equally likely outcomes are HH, HT, TH, TT. The event "two heads" contains one of them, so its probability is 1/4.

If the second flip is instead copied from the first, the model has only HH and TT, each with probability 1/2. Both individual positions still have a 1/2 chance of heads, but the joint probability changes. Matching marginal probabilities does not establish independence.

Conditioning changes the population

Conditional probability P(A|B)=P(A intersection B)/P(B) requires P(B)>0. The denominator restricts attention to outcomes where B occurred. It is not generally symmetric: P(alert|bug) considers buggy cases; P(bug|alert) considers alerted cases.

Events A and B are independent when P(A intersection B)=P(A)P(B). When P(B)>0, this is equivalent to P(A|B)=P(A). Do not multiply probabilities merely because observations occur at different times. Game systems can share state, random streams, players, or workload phases that create dependence.

The complement rule P(not A)=1-P(A) often simplifies calculations. For independent attempts with success probability p, the probability of at least one success in n attempts is 1-(1-p)^n. The independence and fixed-p assumptions are essential; a pity mechanic or cooldown changes the model.

Reverse a conditional with Bayes' rule

For a detector, let p be the prior bug probability, s its sensitivity P(alert|bug), and f its false-positive probability P(alert|no bug). The total alert probability is p*s+(1-p)*f. Bayes' rule gives P(bug|alert)=p*s/[p*s+(1-p)*f].

A high sensitivity does not mean most alerts are true. When true bugs are rare, even a modest false-positive rate can contribute many more alerts from nonbug cases. A contingency table makes the changing denominators visible.

If the total alert probability is zero, the conditional on alert is undefined in this model. Returning zero would pretend a meaningful probability was computed. The reference rejects that situation explicitly.

Simulations check implementations, not assumptions

A seeded simulation can test whether the sampling code approaches a known model probability across many trials. It does not prove that a real detector or real players follow that model. Use synthetic detector values here; they are chosen learning parameters, not measured product claims.

A frequency may differ from the model probability because of ordinary sampling variation, a biased sampler, dependence, or a coding defect. Keep those explanations separate. First validate a small

finite sample space by enumeration. Then compare a larger simulation and document the seed, generator, trial count, and event definitions.

For an engineering report, state the population and time window to which an estimate applies. A test suite's failure rate on deliberately adversarial inputs is not automatically the failure rate across ordinary players. Selection of inputs is part of the probability model, not a minor reporting detail. [R09, R06]

Worked derivation

Imagine 10,000 synthetic test cases. Exactly 100 contain a bug; 9,900 do not. At sensitivity 90%, 90 buggy cases alert and 10 do not. At false-positive rate 5%, 495 nonbug cases alert and 9,405 do not.

There are 585 alerts, only 90 of which are true bugs. Thus P(bug|alert)=90/585=2/13, approximately 0.153846. In contrast, P(alert|bug)=90/100=0.9. Both are correct; they use different denominators.

For two independent attempts with success probability 0.2, the probability of at least one success is 1-0.8^2=0.36. Copying the first outcome to the second would instead leave that probability at 0.2.

Runnable demonstration

Run `python run_example.py 09.1` from the extracted package folder. Predict before executing.

```
from mathkit import posterior_bug
bug_count = 100
nonbug_count = 9900
true_alerts = int(bug_count * 0.9)
false_alerts = int(nonbug_count * 0.05)
print("true and false alerts:", true_alerts, false_alerts)
print(f"P(bug|alert): {posterior_bug(0.01, 0.9, 0.05):.6f}")
print("independent two heads:", 0.5 * 0.5)
print("copied second coin, two heads:", 0.5)
```

Recorded expected output:

```
true and false alerts: 90 495
P(bug|alert): 0.153846
independent two heads: 0.25
copied second coin, two heads: 0.5
```

Practical assignment

Original brief: Simulate independent and dependent binary events. Build a small contingency table for a hypothetical bug detector. Pass: calculate both P(alert|bug) and P(bug|alert) and explain why they differ.

1. Implement posterior_bug(prior,sensitivity,false_positive), validating probability ranges and nonzero alert probability.
2. Construct the full four-cell detector table, including true/false positives and negatives, before using the formula.
3. Simulate independent binary events and a copied-outcome process with the same marginal rates; compare their joint events.

4. Vary only the base rate over 0.001,0.01,0.1 and explain the different posterior probabilities without calling one detector universally accurate.

Coding slice: Work in `labs/student/lab_09_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

An alert rate, sensitivity, false-positive rate, and positive predictive probability are different quantities. Do not swap their denominators or assume independence from temporal separation.

Independent transfer task

Keep sensitivity 0.9 and false-positive rate 0.05, but change the prior bug probability to 0.1. Find P(bug|alert).

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the probability of two independent events with probabilities 0.5 and 0.2 both occurring?
2. Is P(A|B) generally equal to P(B|A)?
3. Why do rare-event base rates matter for an alert?
4. What is the probability of at least one success in three independent trials with p=0.5?
5. When is P(bug|alert) undefined in the detector model?

Assigned study

R09: conditional probability, Bayes theorem, independence, and total probability. R06: finite sample spaces and event counting.

Original reading assignment retained: R09: probability, conditional probability and Bayes' theorem.

Lesson 09.2 — Random variables, distributions and sampling

Learning outcome: Distinguish an expected value from a guaranteed outcome.

A random variable is a numerical outcome

Let reward X be 0, 5, or 20 with probabilities 0.70, 0.25, and 0.05. Its expectation is E[X]=sum(p_i*x_i). It describes an average under the model, not an item that must appear in a single trial. The expected value 2.25 is not even an allowed individual reward here.

Variance is E[(X-E[X])^2], equivalently E[X^2]-E[X]^2. Standard deviation is its square root and has the original units. A mean without a spread can hide very different experiences: a constant reward and a rare large reward can have the same expectation but different variance.

Linearity of expectation does not require independence: E[X+Y]=E[X]+E[Y] whenever the expectations are defined. Adding variances does require care; Var(X+Y)=Var(X)+Var(Y)+2*Cov(X,Y). Independence makes the covariance zero, but simply sharing a mean does not.

Different distributions encode different assumptions

A Bernoulli variable describes one binary trial. A binomial count describes the number of successes in n independent Bernoulli trials with the same probability p; it has mean np and variance np(1-p). A normal distribution is continuous and can approximate some aggregated measurements, but it should not be chosen just because a histogram looks vaguely bell-shaped.

Repeated rewards with a changing probability, a without-replacement draw, or a shared random event do not automatically form a binomial model. Start with the mechanism, then choose the distribution.

A weighted sampler partitions a unit interval

For nonnegative weights, divide by their positive total to obtain probabilities. Cumulative boundaries partition u in [0,1) into intervals. With weights 70,25,5, the intervals are [0,0.70), [0.70,0.95), and [0.95,1). The boundary rule must be explicit: exactly 0.70 belongs to the second interval.

Zero-weight categories have empty intervals and should never be selected. Negative weights, a zero total, and u outside [0,1) are invalid. This bundle separates deterministic weighted_index(weights,u) from the generator producing u. That makes edge cases testable without hoping a random run happens to hit a boundary.

Reproducibility is controlled state

A pseudorandom generator is deterministic given its state and algorithm. A seed helps replay an experiment when the generator and call sequence are controlled. Adding an unrelated random draw to one shared stream can shift every later outcome. Separate streams for unrelated subsystems when that independence of implementation changes is a requirement.

Python's random module is used here for learning simulations, not security-sensitive decisions. Record the interpreter and generator along with the seed; do not promise identical behavior across arbitrary future tool versions or changes to sampling APIs. [R04]

Interpret frequencies at the right scale

For a category probability p and n independent draws, the standard deviation of the observed proportion is sqrt(p(1-p)/n). This is a model calculation, not a guaranteed error bound. Multiplying n by four reduces that standard deviation by two, not four.

A single seeded run is an illustration. Compare several sample sizes and seeds while preserving the planned analysis. Do not rerun until the numbers look impressively close to the expectation and then report only that run. [R09]

Worked derivation

For rewards (0,5,20) and probabilities (0.70,0.25,0.05), the expectation is 0+1.25+1=2.25. E[X^2]=0+25*0.25+400*0.05=26.25, so variance=26.25-2.25^2=21.1875.

The demonstration's 1,000 draws with seed 2026 produce counts [725,220,55] in the tested environment. The observed mean is (220*5+55*20)/1000=2.2. It differs from 2.25 without necessarily indicating a bug.

For the rare category p=0.05 and n=1000, the expected count is 50 and the count standard deviation is sqrt(1000*0.05*0.95), approximately 6.892. The observed count 55 is an observation, not proof that the true probability is 0.055.

Runnable demonstration

Run `python run_example.py 09.2` from the extracted package folder. Predict before executing.

```
import random
from mathkit import weighted_index
rng = random.Random(2026)
weights, rewards = (70, 25, 5), (0, 5, 20)
counts = [0, 0, 0]
for _ in range(1000):
    counts[weighted_index(weights, rng.random())] += 1
print("seed: 2026; trials: 1000")
print("counts:", counts)
expected = sum(w*r for w, r in zip(weights, rewards))/sum(weights)
print(f"model expectation: {expected:.3f}")
print(f"sample mean: {sum(c*r for c,r in zip(counts,rewards))/1000:.3f}")
```

Recorded expected output:

```
seed: 2026; trials: 1000
counts: [725, 220, 55]
model expectation: 2.250
sample mean: 2.200
```

Practical assignment

Original brief: Implement a weighted reward sampler and compare observed frequencies to the intended distribution over several sample sizes. Pass: record seed, generator, counts and expected values; explain finite-sample variation.

1. Implement weighted_index(weights,u), keeping the random generator outside this deterministic function. Test boundaries, zero weights, invalid u, and invalid totals.

2. Simulate the three-category reward model at 100,1000,10000 draws and at least three recorded seeds.
3. Derive the expectation and variance by hand; compare empirical counts and means without demanding exact frequency matches.
4. Demonstrate how an extra draw changes a shared stream, then show two separate generator instances preserve the unrelated subsystem sequence.

Coding slice: Work in `labs/student/lab_09_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

An expected value is not a per-trial guarantee. Setting a seed does not establish statistical independence, unbiased sampling, or security.

Independent transfer task

If two rewards are perfectly copied from one another, each with variance s^2, what is the variance of their sum?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the expectation of 20 with probability 0.25 and 0 otherwise?
2. Does an expectation of 5 guarantee a reward of 5 on a trial?
3. Why separate random streams for unrelated systems?
4. Under the specified weighted intervals, which category contains u=0.70 for weights (70,25,5)?
5. Does linearity of expectation require independent variables?

Assigned study

R09: random variables, expectation, variance, Bernoulli/binomial models, and sampling. R04: random.Random, seeding, and reproducibility limitations.

Original reading assignment retained: R09: random variables, expectation and variance; R04: random reference.

Lesson 09.3 — Inference, uncertainty and benchmark design

Learning outcome: Compare implementations without hiding variance or bias.

Name the estimand and the experimental unit

An estimand is the quantity you intend to learn: mean update time per independent run, p99 frame latency for a workload, or the mean paired difference between implementations. These are not interchangeable. Decide which one matters before collecting data.

A frame inside a long run is often correlated with nearby frames through workload phases, caches, scheduling, or temperature. Counting every frame as an independent experiment can make uncertainty look much smaller than justified. A run or a randomized paired block may be the more defensible experimental unit, but independence is still an assumption to assess.

Summaries reveal different behavior

The mean adds values and divides by their count. The median gives a central order statistic and is less affected by a few large values. A tail percentile asks where a specified fraction of observed values lies. Different interpolation conventions can produce different sample percentiles, so this bundle defines the nearest-rank rule explicitly.

For n sorted observations and p in (0,1], choose the element at one-based rank ceil(p*n). For p=0, use the minimum. With only twenty samples, p99 is simply the maximum; it is not a precise estimate of a population tail. Always report sample size and the convention.

Pair comparable runs

To compare A and B, use the same declared workload and calculate d_i=B_i-A_i within each pair. Positive differences mean B took longer. Randomize whether A or B runs first within each pair to reduce order-related confounding. Warm up both, exclude only according to a predefined rule, verify equivalent results, and record every raw pair.

Separate per-operation time from whole-frame time. A faster helper function does not establish a faster game if it is a tiny fraction of the frame or changes the work performed. State whether allocation, setup, validation, rendering, and I/O are included.

Confidence intervals are conditional procedures

For independent normally distributed paired differences with unknown variance, a Student t interval for their mean uses mean(d) plus or minus t_critical*s_d/sqrt(n), with n-1 degrees of freedom. The reference mean_interval routine accepts the critical value explicitly; it does not infer a distribution, select a test, or verify independence.

The 95% interpretation concerns coverage across repeated datasets generated under the model. A realized frequentist interval is not a 95% probability statement about a fixed unknown parameter. Model violations, biased input selection, and cherry-picked stopping can invalidate the nominal coverage.

For five differences, the illustrative two-sided 95% t critical value is approximately 2.776445. This is a small teaching case; it is not a recommendation to use five runs for every performance study.

Statistical evidence is not practical importance

A p-value measures how unusual a statistic at least as extreme as observed would be under a specified null model. It is not the probability the null hypothesis is true, nor a measure of effect size. A tiny statistically detectable improvement can be operationally irrelevant; a rare visible pause can matter despite a small change in the mean.

Report differences, uncertainty assumptions, tails, and practical requirements. Preserve negative results and do not stop only when the result crosses a preferred threshold. [R09]

Worked derivation

Consider the synthetic frame list containing nineteen values of 5 ms and one value of 100 ms. Its mean is (95+100)/20=9.75 ms. Under nearest-rank percentiles, p95 has rank 19 and equals 5 ms; p99 has rank 20 and equals 100 ms. The outlier is real within the synthetic example, but p95 alone misses it.

For illustrative paired differences [1,2,3,4,5] ms, the mean is 3 ms. The sample variance is 10/4=2.5 ms^2, and the standard error is sqrt(2.5/5)=sqrt(0.5) ms. With critical value 2.776445, the interval is approximately [1.037,4.963] ms, under the stated independent-normal model.

Runnable demonstration

Run `python run_example.py 09.3` from the extracted package folder. Predict before executing.

```
from mathkit import percentile, mean_interval
frames = [5.0]*19 + [100.0]  # synthetic, not a measured machine
print(f"synthetic mean: {sum(frames)/len(frames):.3f} ms")
print("nearest-rank p95 and p99:", percentile(frames, 0.95), percentile(frames, 0.99))
# Five illustrative independent normal run differences; B minus A in ms.
mean, low, high = mean_interval([1, 2, 3, 4, 5], 2.776445)
print(f"paired mean and illustrative t interval: {mean:.3f} [{low:.3f}, {high:.3f}]")
```

Recorded expected output:

```
synthetic mean: 9.750 ms
nearest-rank p95 and p99: 5.0 100.0
paired mean and illustrative t interval: 3.000 [1.037, 4.963]
```

Practical assignment

Original brief: Compare two implementations using the same inputs, randomized test order and repeated runs. Pass: report median, a tail percentile, sample count and limitations; distinguish observations from causal claims.

1. Implement percentile(values,p) with the nearest-rank convention, and mean_interval(values,critical) using sample variance with denominator n-1.
2. Run the included benchmark runner or two independently implemented equivalent operations. Preserve randomized pair order, warmup, raw samples, and the workload.

3. Report mean, median, p95 or p99 with their convention and sample count, plus paired differences and clearly qualified uncertainty.
4. Analyze the synthetic tail example separately. Never relabel its chosen numbers as measured hardware performance.

Coding slice: Work in `labs/student/lab_09_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not treat correlated frames as independent trials, label a sample maximum a well-estimated p99, or interpret statistical significance as a useful speedup.

Independent transfer task

A is 5 ms and B is 4 ms in each paired run. What sign does B-A have, and what does that sign mean?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Why can adjacent frame times violate an independence assumption?
2. Does a small p-value establish a large useful improvement?
3. Why report a tail percentile alongside a mean?
4. For nineteen 5 ms samples and one 100 ms sample, what is nearest-rank p95?
5. What does a p-value not tell you about an optimization?

Assigned study

R09: confidence intervals, Student t models, hypothesis tests, and experimental reasoning. The supplied five-point t example states its distributional assumptions explicitly.

Original reading assignment retained: R09: confidence intervals, hypothesis testing and experimental reasoning.

Lesson 09.4 — Regression, correlation and ethical telemetry

Learning outcome: Fit a model while recognizing confounding and data limits.

A fitted relationship has a scope

A regression model predicts a response from specified inputs. For update time versus entity count, a simple model is time approximately a*count+b. The intercept may represent fixed overhead and the slope incremental work, but those interpretations require a controlled experiment and an adequate model. A fitted coefficient is not automatically a physical constant.

Least squares minimizes the squared residuals on the fitting data. It does not guarantee good prediction on new workload families, different machines, or larger counts outside the tested range. Cache transitions, changing algorithms, and scheduling can make the relationship nonlinear or piecewise.

Separate fitting from evaluation

Choose training and held-out sets before examining the desired final metric. Fit a and b on training observations only, then evaluate predictions against the held-out responses. Changing the model repeatedly based on the held-out set turns it into tuning data; a final untouched test set or a clearly disclosed validation strategy is then needed.

For correlated observations, split at an appropriate group level. Putting adjacent frames from the same run into both training and test sets can leak workload-specific information and exaggerate generalization. Holding out entire runs, scenes, or count regimes can answer a more meaningful transfer question.

Define and interpret prediction error

A residual is observed minus predicted. Mean squared error averages squared residuals; root mean squared error (RMSE) takes the square root and returns to the response's units. Mean absolute error measures a different objective and is less dominated by a single large residual. A good report includes residual patterns, not only one aggregate number.

Compare with a simple baseline, such as predicting the training-set mean for every held-out observation. A complicated model should earn its complexity with relevant evidence. Report both interpolation within the training range and extrapolation outside it as distinct cases.

Correlation is not a causal explanation

Two variables can move together because of a shared cause. Scene complexity might raise both entity count and GPU time. Hardware class might influence both settings choices and measured performance. A regression using only one observed predictor can incorrectly attribute the whole association to that predictor.

A controlled benchmark that deliberately changes entity count while holding other relevant factors fixed gives stronger evidence for that intervention within the test conditions. It still does not establish

every real game's behavior. Randomization helps with some confounders, but does not repair a measurement that omits the main work or compares incorrect implementations.

Collect only data needed for the question

The supplied datasets are synthetic or local benchmark outputs. They contain no real player identifiers. For a course experiment, entity count, trial ID, algorithm, duration, environment version, and correctness status are usually enough. Do not add names, account identifiers, precise locations, chat logs, or unrelated activity simply because a telemetry system could collect them.

Write a short data plan stating purpose, fields, retention, who can read the files, and how results will be shared. A filename or machine identifier can itself expose unnecessary information; the runner avoids collecting a username or hostname. Professional measurement includes provenance and appropriate boundaries, not only statistical formulas. [R09; course data-minimization policy]

Worked derivation

Use synthetic training pairs x=(1,2,3,4), y=(3,5,7,9). They lie exactly on y=2x+1, so the training residuals are zero. Hold out x=(5,6), with observed y=(11,14). Predictions are (11,13), so residuals are (0,1).

The held-out RMSE is sqrt((0^2+1^2)/2)=sqrt(0.5), approximately 0.707107 response units. Zero training error did not imply zero held-out error.

The training-mean baseline predicts 6 for each held-out point. Its RMSE is sqrt(((11-6)^2+(14-6)^2)/2)=sqrt(44.5), approximately 6.671. The line is better on this chosen test, but the small synthetic dataset does not establish a universal performance law.

Runnable demonstration

Run `python run_example.py 09.4` from the extracted package folder. Predict before executing.

```
from mathkit import fit_line, rmse
train_x, train_y = (1, 2, 3, 4), (3, 5, 7, 9)
slope, intercept = fit_line(train_x, train_y)
test_x, test_y = (5, 6), (11, 14)  # synthetic held-out observations
predicted = [slope*x + intercept for x in test_x]
print("fitted slope and intercept:", slope, intercept)
print("held-out predictions:", predicted)
print(f"held-out RMSE: {rmse(test_y, predicted):.6f}")
```

Recorded expected output:

```
fitted slope and intercept: 2.0 1.0
held-out predictions: [11.0, 13.0]
held-out RMSE: 0.707107
```

Practical assignment

Original brief: Fit a simple performance model using entity count and measured update time. Hold out some workloads. Pass: report prediction error and one confounder; write a short data-minimization plan for the experiment.

1. Implement rmse(actual,predicted) for nonempty equal-length finite sequences. Reuse the independently tested line-fitting routine.

2. Fit update time versus entity count from declared local measurements; reserve at least one workload group before fitting. The included synthetic CSV is practice only.
3. Compare the fitted model to a training-mean baseline, preserve residuals, and distinguish interpolation from extrapolation.
4. Write a data-minimization and provenance note, identifying one possible confounder and which claims the experiment does not support.

Coding slice: Work in `labs/student/lab_09_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Perfect training fit is not independent evaluation. A correlated predictor is not necessarily the cause, and synthetic timings must not be reported as observed machine results.

Independent transfer task

Why is randomly splitting adjacent frames from one run into training and test sets potentially misleading?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Why is training-set fit alone insufficient?
2. What is a confounder?
3. Why should telemetry collect only what the question requires?
4. Predictions are [2,4] and observations [3,4]. What is RMSE?
5. What simple baseline should accompany a fitted performance model?

Assigned study

R09: regression, inference, and model evaluation. The privacy/data-minimization requirements are course design rules, not a legal-compliance certification.

Original reading assignment retained: R09: regression and inference; R01: ethics and data-management breadth.

Module 10 — Signals, optimization and numerical foundations

Original prerequisites: Modules 07-09.

Original module project: Complete Gate B: a mathematical engineering notebook plus tested transforms, numerical integration, signal filtering and an uncertainty-aware experiment.

Expanded project: Gate B: integrated game-mathematics notebook. The accumulated projects plus a sampling/filtering demonstration, a gradient check, and a convergent/nonconvergent optimization comparison, assembled into one reproducible handoff.

Acceptance: Satisfy CAPSTONE_SPEC.pdf: proofs, tested transforms, analytic dynamics/refinement, aliasing/filtering, probability checks, a measured experiment, data provenance, and an independent reproduction. These reuse module work; they are not five extra unrelated capstones.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 10.1 — Multivariable calculus and gradients

Learning outcome: Use partial derivatives to reason about spatial fields and optimization.

Change one input at a time

A scalar field f(x,y) maps a two-component input to one number, such as a cost or height. Its partial derivative with respect to x changes x while holding y fixed. The gradient collects these partial derivatives. For f=x^2+3y^2, grad(f)=(2x,6y).

The gradient points toward steepest local increase under the ordinary Euclidean metric. Its magnitude and direction depend on coordinate scaling and units. Replacing meters by millimeters changes numerical derivatives even when the underlying problem is equivalent. Normalize or scale parameters deliberately rather than treating arbitrary component magnitudes as physically comparable.

For a unit direction u, the directional derivative is grad(f)·u. Without a unit direction, that dot product includes the magnitude of the proposed input change. The first-order approximation is f(p+delta) approximately f(p)+grad(f)(p)·delta.

A Jacobian generalizes the derivative

For a vector-valued mapping F from n inputs to m outputs, the Jacobian is an m-by-n matrix with entry Jij=partial Fi/partial xj. It maps a small input displacement into an approximate output displacement. A gradient is associated with a scalar output; a Jacobian can represent an entire coordinate transformation or multi-output constraint.

For F(x,y)=(x^2+y,xy), J=[[2x,1],[y,x]]. At (2,3), J=[[4,1],[3,2]]. Applying it to delta=(0.01,-0.02) predicts delta_F=(0.02,-0.01). The true finite change also contains higher-order terms, so exact equality is not expected for a nonzero step.

Follow the chain rule through every transform

For a scalar cost g(F(x)), the gradient with respect to x is J_F(x)^T*grad(g)(F(x)) in this column-vector convention. The transpose appears because each input can influence several outputs. Omitting a transform or applying the Jacobian in the wrong direction can produce a vector of the correct length but the wrong meaning.

This is the mathematical foundation of gradient-based fitting and, later, reverse-mode differentiation. It is useful to derive a two-variable case manually before depending on an automatic-differentiation tool.

Stationary does not always mean minimum

A stationary point has zero gradient. For f=x^2+y^2, the origin is a minimum. For f=-(x^2+y^2), it is a maximum. For f=x^2-y^2, it is a saddle: some directions increase the value and others decrease it. The Hessian, a matrix of second partial derivatives, helps classify local curvature under suitable smoothness assumptions.

Constraints can also make a boundary point optimal even when its unconstrained gradient is nonzero. A zero-gradient stopping rule is appropriate only for the optimization problem actually being solved.

Check derivatives, but do not confuse a check with a proof

Centered differences perturb one coordinate at a time while holding others fixed. The reference uses the same h for each coordinate as a transparent educational choice. For strongly different scales, choose scale-aware perturbations and report them. Test several h values and include stationary points and nonzero points.

A smooth analytic gradient and a finite difference should agree within an error budget where truncation and roundoff are controlled. They can disagree because of a coding defect, a nonsmooth operation, a bad perturbation scale, or inconsistent coordinate conventions. A single matching point is weak evidence.

For a y-up terrain represented as y=h(x,z), a convenient unnormalized surface normal is (-partial h/partial x,1,-partial h/partial z). This explicitly matches the geometry convention rather than silently treating z as up. [R41, R07]

Worked derivation

For f(x,y)=x^2+3y^2 at (2,-1), the gradient is (4,-6) and the objective is 7. With step size alpha=0.1, a negative-gradient step gives (1.6,-0.4). The new objective is 1.6^2+3*(-0.4)^2=3.04.

This proves descent for the chosen step by direct evaluation; it does not prove that every step size works. At a sufficiently large step, the same correct gradient can increase the objective.

For the mapping F in the lesson, the exact change from (2,3) to (2.01,2.98) is (0.0201,-0.0102). The Jacobian prediction (0.02,-0.01) differs by second-order terms.

Runnable demonstration

Run `python run_example.py 10.1` from the extracted package folder. Predict before executing.

```
from mathkit import gradient
f = lambda p: p[0]**2 + 3*p[1]**2
point = (2.0, -1.0)
g = gradient(f, point, 0.0001)
print("numerical gradient:", tuple(round(x, 6) for x in g))
print("analytic gradient:", (2*point[0], 6*point[1]))
alpha = 0.1
new = tuple(x-alpha*dx for x, dx in zip(point, g))
print(f"objective: {f(point):.3f} -> {f(new):.3f}")
```

Recorded expected output:

```
numerical gradient: (4.0, -6.0)
analytic gradient: (4.0, -6.0)
objective: 7.000 -> 3.040
```

Practical assignment

Original brief: Derive gradients for three functions and check them with centered differences. Pass: test nonzero and stationary points, vary perturbation size and report errors rather than claiming exact equality.

1. Implement gradient(f,point,h), perturbing one coordinate at a time, rejecting invalid/unresolvable h and nonfinite evaluations.

2. Derive and check three gradients, including a coupled function such as x*y+x^2 and a function with a stationary saddle.
3. Derive the Jacobian example and compare predicted versus exact finite changes over several step magnitudes.
4. Create a y-up terrain-normal calculation and explicitly document which variables parameterize the surface.

Coding slice: Work in `labs/student/lab_10_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

A gradient is local and coordinate-dependent. A zero gradient can describe a saddle or maximum, and a correct gradient does not make every step size safe.

Independent transfer task

For the y-up height field h(x,z)=x^2+z^2, find a unit surface normal at (x,z)=(1,0).

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the gradient of x*x + 3*y*y?
2. What does a Jacobian describe?
3. Does a zero gradient always identify a minimum?
4. What is the Jacobian of F(x,y)=(x^2+y,xy)?
5. Classify the origin for f=x^2-y^2.

Assigned study

R41: partial derivatives, tangent approximations, chain rule, gradients, and directional derivatives. R07: linear maps and transposes. Complete the multivariable bridge before relying on this material in graduate optimization.

Original reading assignment retained: R07: linearization foundations; R23: geometric differential quantities; use the program's multivariable extension reading.

Lesson 10.2 — Sampling, aliasing and Fourier ideas

Learning outcome: Explain why discrete samples can misrepresent a continuous signal.

Samples do not uniquely determine every continuous signal

Sampling at rate fs observes a signal at t_n=n/fs. A sinusoid with frequency f has discrete phase increments 2*pi*f/fs. Adding an integer multiple of fs to its frequency adds an integer number of complete revolutions at every sample, leaving those sample values unchanged.

High-frequency content can therefore appear as a different lower-frequency signal. This is aliasing, not random numerical noise. A baseband alias magnitude can be found by reducing the frequency modulo fs and reflecting values above fs/2. That magnitude alone does not preserve phase or the sign relationship between sampled sines.

For cosine, frequencies 900 Hz and 100 Hz sampled at 1000 Hz give the same samples because cos(2*pi*0.9n)=cos(2*pi*n-2*pi*0.1n)=cos(2*pi*0.1n). For sine the corresponding relation has a minus sign. Saying merely "900 becomes positive 100" would lose that phase information.

Bandwidth assumptions make reconstruction possible

An ideal sampling theorem applies to appropriately band-limited signals sampled above twice their highest frequency. The half-rate fs/2 is the Nyquist boundary. A sinusoid exactly at that boundary can lose phase information; for example, sin(pi*n)=0 for every integer n. Practical filters need transition bands and margin, not only an equality in a specification.

Increasing sample rate helps relative to the signal bandwidth. It does not justify sampling arbitrary sharp edges without filtering. Similarly, a high-resolution texture can still alias when it is reduced to only a few screen pixels.

Filter before discarding samples

Downsampling keeps fewer samples, lowering the representable bandwidth. A low-pass filter applied first suppresses components that would otherwise fold into misleading lower frequencies. Filtering after aliasing has occurred cannot generally identify which original signal produced the samples.

The checker sequence [0,1,0,1,...] illustrates this. Keeping every other value can produce all zeros or all ones depending on phase. Averaging adjacent pairs before keeping one value yields 0.5 for this exact pattern. The two-tap average solves that chosen example; it is not an ideal anti-alias filter for every possible signal.

Fourier components provide another view

Complex numbers introduce i with i^2=-1. Euler's identity exp(i*theta)=cos(theta)+i*sin(theta) packages phase conveniently. A discrete Fourier transform of N samples is X[k]=sum_n x[n]*exp(-i*2*pi*k*n/N). Under this convention the inverse has a 1/N factor and a positive exponent.

Bin k corresponds to k*fs/N before accounting for the wrapped negative-frequency interpretation of upper bins. Real signals have conjugate symmetry. A finite observation window can spread a sinusoid's energy across bins when it does not align with an integer number of cycles; this is spectral leakage, distinct from aliasing.

For x=[1,0,-1,0], the unnormalized four-point DFT is [0,2,0,2]. The two nonzero bins describe the positive and negative parts of a real cosine. A faster FFT computes the same transform family; it is not a different physical model.

Keep time and space analogous but explicit

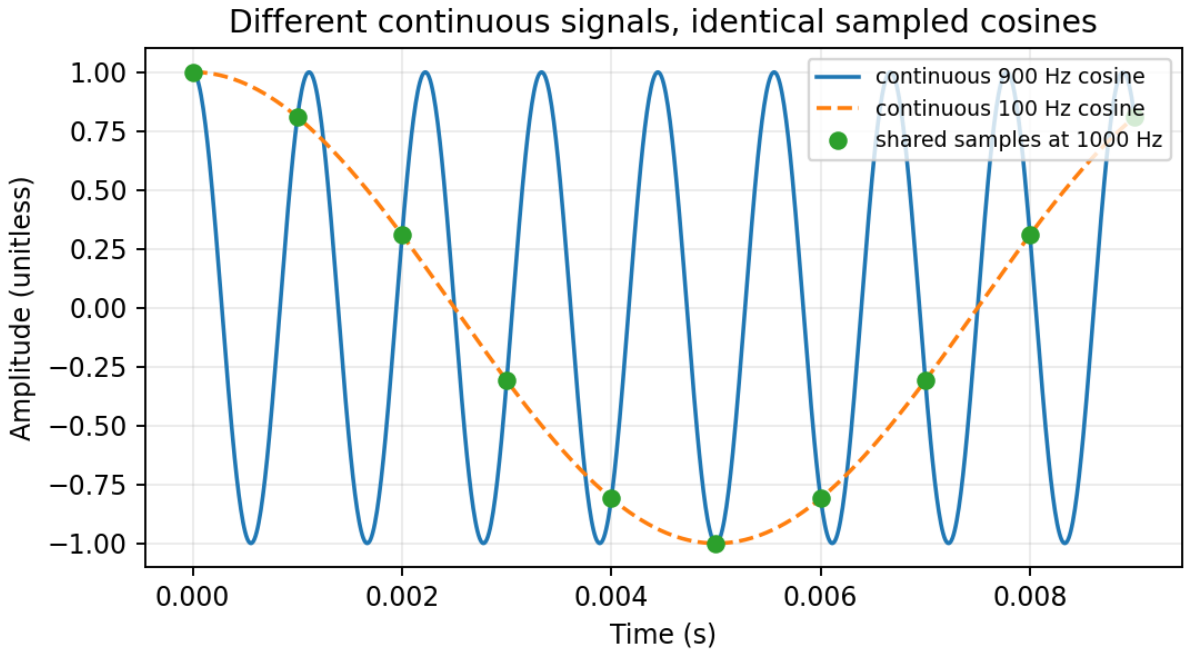
Audio samples are spaced in time, texture samples in space, and animations may be sampled in both. The same sampling reasoning applies, but their units and reconstruction assumptions differ. Record sample rate or spacing, frequency units, filter, boundary policy, and decimation phase. [R10, R23]

Worked derivation

At fs=1000 Hz, the samples of cos(2*pi*900*t) match cos(2*pi*100*t). The demonstration evaluates ten sample indices and checks that their maximum numerical disagreement is below 1e-12 in the tested environment. The trigonometric identity, not those ten checks alone, explains all integer sample indices in exact arithmetic.

For checker=[0,1,0,1,0,1,0,1], raw decimation starting at index zero gives [0,0,0,0]. Adjacent pair averages give [0.5,0.5,0.5,0.5]. Starting the raw decimation at index one would instead yield all ones.

At 48,000 samples/second, the Nyquist boundary is 24,000 Hz. It is a boundary in an idealized analysis, not a guarantee that a practical filter passes everything up to it perfectly.



Continuous cosines differ between samples, but the marked 1000 Hz samples coincide for the chosen 900 Hz and 100 Hz cosines.

Runnable demonstration

Run `python run_example.py 10.2` from the extracted package folder. Predict before executing.

```
import math
from mathkit import alias_frequency
print("alias magnitude:", alias_frequency(900, 1000), "Hz")
# Cosines match; a 900 Hz sine would be the NEGATIVE of the 100 Hz sine.
errors = [abs(math.cos(2*math.pi*900*n/1000) - math.cos(2*math.pi*100*n/1000)) for n in range(10)]
print("cosine samples agree within 1e-12:", max(errors) < 1e-12)
checker = [0, 1]*4
print("raw decimation:", checker[::2])
print("pair-averaged decimation:", [(checker[i]+checker[i+1])/2 for i in range(0, len(checker),
    2)])
```

Recorded expected output:

```
alias magnitude: 100.0 Hz
cosine samples agree within 1e-12: True
raw decimation: [0, 0, 0, 0]
pair-averaged decimation: [0.5, 0.5, 0.5, 0.5]
```

Practical assignment

Original brief: Generate low- and high-frequency sine samples, then downsample a checker pattern with and without prefiltering. Pass: explain the artifacts using frequency content rather than calling them random noise.

1. Implement alias_frequency(frequency,sample_rate), returning only the nonnegative baseband magnitude and documenting that phase is not encoded.
2. Generate the 900/100 Hz cosine pair and the corresponding signed sine relation at fs=1000. Verify both analytically and numerically.
3. Downsample a checker pattern with both starting phases and with a declared prefilter. Do not hide the filter delay or boundary behavior.
4. Compute the four-point DFT example by hand and explain why this is a different issue from the alias demonstration.

Coding slice: Work in `labs/student/lab_10_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not confuse aliasing with random noise or spectral leakage, omit the sine phase inversion, or claim a two-tap average is a universal reconstruction filter.

Independent transfer task

A cosine at 700 Hz is sampled at 1000 Hz. Find its baseband alias magnitude and explain a sine caveat.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the Nyquist frequency at 48,000 samples/second?
2. Why filter before downsampling?

3. Can post-filtering always undo aliasing?
4. Why is sampling exactly at twice a sinusoid frequency risky for phase information?
5. What distinguishes spectral leakage from aliasing?

Assigned study

R10: sampling and Fourier representations. R23: sampling and reconstruction. The complex-number and DFT conventions here are an explicit Package 03 bridge, not an additional numbered module.

Original reading assignment retained: R10: sampling and Fourier representation; R23: Sampling and Reconstruction.

Lesson 10.3 — Convolution, filters and systems

Learning outcome: Predict a filter's effect on data and latency.

A filter is a mathematical operation with boundaries

For discrete signal x and kernel h, convolution is y[n]=sum_k h[k]*x[n-k]. The n-k indexing shifts and reverses the kernel relative to the input. A routine using x[n+k] without a corresponding reindexing generally computes a correlation-like operation instead.

This bundle defines finite full convolution with zero extension outside the input. If the signal has N values and the nonempty kernel has K values, the output has N+K-1 values. An empty signal produces an empty output. Other libraries may use "same" or "valid" modes with different alignment; those names do not remove the need to specify the contract.

Derive a system from its impulse response

A unit impulse is one at index zero and zero elsewhere. A discrete signal can be written as a sum of shifted impulses weighted by its samples. For a linear time-invariant system, each shifted impulse produces a correspondingly shifted response, and superposition adds the weighted responses. That yields convolution with the impulse response.

Linearity means preserving weighted sums. Time invariance means an input shift produces the same output shift. A clamp is nonlinear; a filter whose coefficients change each frame is generally not time-invariant. Do not assume the impulse-response model describes every game subsystem.

Smoothing changes delay and detail

A three-sample moving average has coefficients (1/3,1/3,1/3). In a causal implementation it uses the current and two previous samples. It suppresses some rapid variation but also rounds edges and delays features. A symmetric length-K causal finite-impulse-response filter has linear phase with group delay (K-1)/2 samples where its frequency response is nonzero.

For K=3 at fs=1000 samples/second, that delay is one sample, or 1 ms. It is only the filter's contribution, not total controller-to-display latency. A centered offline filter may align the result differently, but it uses future samples unavailable to a real-time causal system.

The sum of coefficients explains constant signals

For a constant infinite signal c, convolution gives c*sum(h). A unity-sum kernel therefore preserves the constant in the interior or steady-state region. At finite boundaries with zero extension, the kernel overlaps missing samples and the output can ramp or fall. That is expected under the chosen boundary policy.

Zero padding, edge replication, reflection, and wrapping represent different assumptions about data outside the recorded region. A wrapped terrain texture may need a different policy from an audio segment that starts in silence. Make the choice explicit and test it.

Numeric and implementation checks

Test an impulse to reveal the kernel and its alignment. Test a constant to reveal gain and boundary effects. Test a step to reveal smoothing and delay. Use an asymmetric kernel as well as a symmetric one, because reversing a symmetric kernel can hide a convolution-versus-correlation error.

Every input and coefficient in the exercise must be finite. Large coefficients can amplify noise and overflow an implementation even when the formula is well defined in real arithmetic. A short direct convolution has cost proportional to N*K; FFT-based alternatives have different overhead and padding requirements, and are not automatically better for a tiny kernel.

When filtering colors, the numerical values must be interpreted in the intended space; averaging encoded display values is not automatically the same operation as averaging linear light. That detailed rendering issue returns in Volume 5. Here the scalar signal's meaning and units are declared directly. [R10, R23]

Worked derivation

For signal [1,2,3] and kernel [0.5,0.5], zero-extended full convolution is [0.5,1.5,2.5,1.5]. At n=1, the calculation is 0.5*x[1]+0.5*x[0]=1.5. At the last output, only the final input overlaps the kernel.

For signal [1], the output equals the kernel exactly, showing the impulse response. For a constant four-sample input [1,1,1,1], the same kernel produces [0.5,1,1,1,0.5]. Taking the first four values gives a causal same-length prefix; the initial 0.5 is the declared zero-history transient.

With asymmetric kernel [1,2] and signal [1,2,3], full convolution gives [1,4,7,6]. Keep this as a test that a symmetric moving average cannot replace.

Runnable demonstration

Run `python run_example.py 10.3` from the extracted package folder. Predict before executing.

```
from mathkit import convolve
print("full convolution:", convolve([1, 2, 3], [0.5, 0.5]))
print("impulse response:", convolve([1], [0.25, 0.5, 0.25]))
x = [1, 1, 1, 1]
print("causal same-length prefix:", convolve(x, [0.5, 0.5])[:len(x)])
```

Recorded expected output:

```
full convolution: [0.5, 1.5, 2.5, 1.5]
impulse response: [0.25, 0.5, 0.25]
causal same-length prefix: [0.5, 1.0, 1.0, 1.0]
```

Practical assignment

Original brief: Implement a one-dimensional convolution with documented boundary handling. Test an impulse, a constant and a step. Pass: predict the output before running and identify the filter's delay convention.

1. Implement convolve(signal,kernel) as full zero-extended convolution. Return [] for empty signal, reject empty kernel and nonfinite data.
2. Predict and test impulse, constant, step, and asymmetric-kernel outputs. Include output length and alignment checks.

3. Compare full output with a causal same-length prefix and with a separately specified boundary alternative.
4. Explain gain from sum(kernel), the filter delay in samples and seconds, and why a future-looking centered filter is different in real time.

Coding slice: Work in `labs/student/lab_10_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Boundary handling and output alignment change the result. Unity-sum coefficients preserve a constant in steady state, not necessarily at zero-padded finite edges.

Independent transfer task

A symmetric five-tap causal filter runs at 1000 Hz. What is its linear-phase group delay, and what does it exclude?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What does convolving with an impulse reveal?
2. What is the average of 2, 5 and 8?
3. Why is boundary handling not an implementation footnote?
4. What is the full convolution output length for N=4 and K=3, under this bundle contract?
5. Why test an asymmetric kernel?

Assigned study

R10: convolution, impulse and step responses, linear time invariance, and frequency response. R23: later image-filtering context.

Original reading assignment retained: R10: convolution, impulse response and linear time-invariant systems.

Lesson 10.4 — Optimization, conditioning and approximation budgets

Learning outcome: Choose an objective and distinguish convergence from correctness.

Choose what is being optimized

An optimization problem has variables, an objective, and constraints. "Make the game better" is not a mathematical objective. Minimizing a fitting error, minimizing time while keeping error below a threshold, and minimizing memory under a latency budget are different problems.

A local minimum need not be global. Convex objectives over convex feasible regions have useful global properties, but those assumptions must be established rather than inferred from a smooth-looking graph. This lesson begins with a quadratic whose behavior can be derived exactly.

Derive fixed-step gradient descent

For f(x)=(x-3)^2, the derivative is 2(x-3). A gradient step is x_new=x-alpha*2(x-3). Define error e=x-3; then e_new=(1-2alpha)*e. Repeating gives e_n=(1-2alpha)^n*e_0.

The error decays exactly when |1-2alpha|<1, giving 0<alpha<1. At alpha=0.5 the exact-arithmetic update reaches the minimizer in one step. At alpha=1 it alternates with unchanged error magnitude; above one it diverges except from the exact minimizer. A correct gradient is compatible with a bad step-size choice.

For f(x,y)=(x-3)^2+3(y+1)^2, the two error factors are 1-2alpha and 1-6alpha. Both must have magnitude below one, so 0<alpha<1/3. The steeper direction imposes the tighter condition. This connects curvature, scaling, and the eigenvalues studied earlier.

Stop with a declared status

A useful solver reports its achieved point, objective, gradient or residual, iteration count, and reason for stopping. Reaching an iteration limit is not convergence. A nonfinite result is not an acceptable approximate answer and should trigger an explicit failure path.

A small step norm is also not enough by itself: an extremely small alpha can make progress tiny far from the solution. For this particular one-dimensional quadratic, |f'(x)|<=tol implies |x-3|<=tol/2. That bound follows from the known derivative and does not automatically apply to arbitrary objectives.

The reference uses gradient tolerance and an iteration cap. It retains history for inspection and raises an arithmetic error on a nonfinite trajectory rather than emitting a false success record.

Constraints can change the optimality condition

With the added constraint x>=4, the minimizer of (x-3)^2 is x=4, where the unconstrained gradient is 2, not zero. Projecting a proposed point back into an interval is one possible algorithmic building block, but it changes the convergence analysis. Detailed constrained optimality and multipliers are reserved for the later graduate sequence.

In engineering, a quality limit is often a hard constraint. Adding a small penalty for violating it does not guarantee the limit will be respected. Check feasibility independently of the objective value.

Budget approximation error rather than hiding it

An application can accumulate model, discretization, sampling, roundoff, and iterative-solver errors. If defensible worst-case absolute bounds apply to quantities in the same output units, the triangle inequality permits adding them conservatively. Root-sum-of-squares aggregation needs additional stochastic assumptions; it is not a universal shortcut.

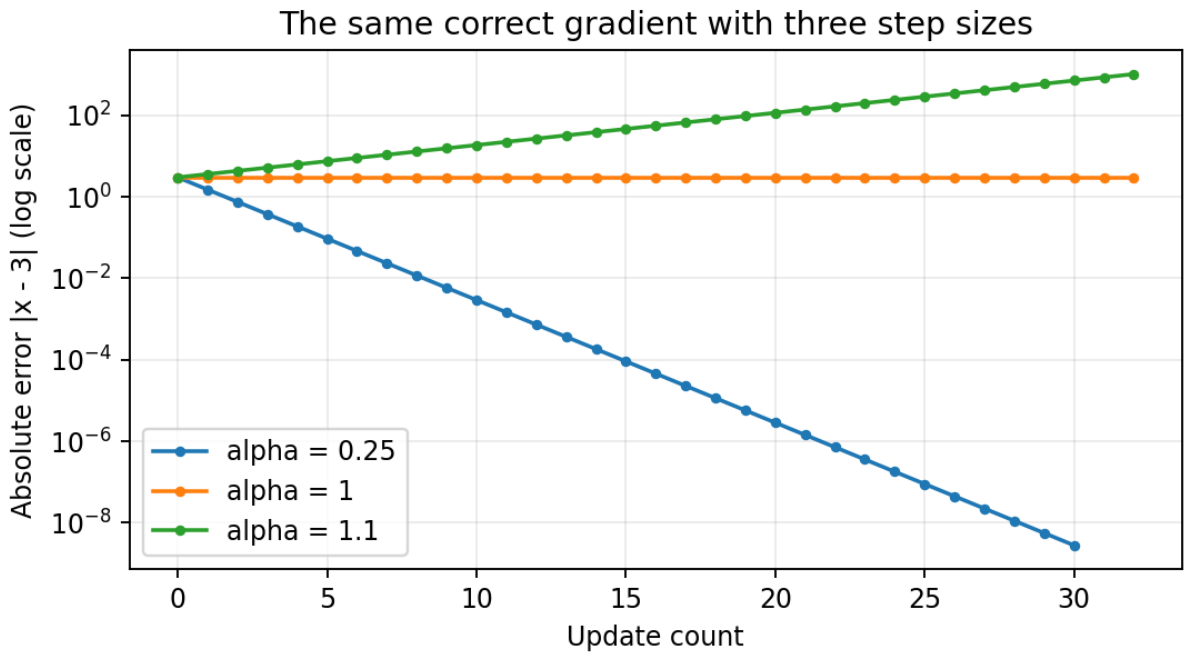
Identify a reference case, state which errors are controlled, and record the remaining uncertainty. More solver iterations reduce only the error the solver is responsible for. They cannot fix an incorrect model or aliased input. Completing Gate B means defending those distinctions with equations, tests, and reproducible data. [R42, R07; quadratic derivations in this lesson]

Worked derivation

Start at x0=0, so e0=-3. At alpha=0.25, the error factor is 0.5. After eight updates, x=3-3/256=2.98828125 and |gradient|=0.0234375. A tolerance of 0.03 is met.

At alpha=1, the error factor is -1, so the iterates alternate 0,6,0,6,... and do not converge from this start. At alpha=1.1, the factor is -1.2; after eight updates x=3-3*(1.2)^8, approximately -9.899451. The reference reports the iteration limit, not success.

Suppose known absolute output-error bounds are 0.01,0.02,0.005 in the same units. A conservative combined bound is 0.035. Calling sqrt(0.01^2+0.02^2+0.005^2) a guaranteed bound would require assumptions not supplied here.



Absolute scalar-quadratic error for selected step sizes. The alpha=1 trace oscillates in signed error while its magnitude stays fixed.

Runnable demonstration

Run `python run_example.py 10.4` from the extracted package folder. Predict before executing.

```
from mathkit import descent_quadratic
for alpha in (0.25, 1.0, 1.1):
    result = descent_quadratic(0, alpha, max_steps=8, tol=0.03)
    print(f"alpha={alpha:.2f} x={result['x']:.6f} steps={result['iterations']}
    reason={result['reason']}")
```

Recorded expected output:

```
alpha=0.25 x=2.988281 steps=8 reason=gradient_tolerance
alpha=1.00 x=0.000000 steps=8 reason=iteration_limit
alpha=1.10 x=-9.899451 steps=8 reason=iteration_limit
```

Practical assignment

Original brief: Implement gradient descent on a quadratic and a two-variable problem. Pass: compare step sizes, record residuals and state a stopping rule; complete Gate B with no unsupported claim of global optimality.

1. Implement descent_quadratic(x0,alpha,max_steps,tol) with explicit convergence/iteration-limit status, achieved gradient, and a history of iterates.
2. Compare alpha=0.25,0.5,1,1.1 from the same start. Preserve nonconvergent cases and verify reported status at the final allowed iteration.
3. Implement a two-variable extension and derive its step-size restriction from the two error factors, rather than guessing by trial alone.
4. Complete Gate B: assemble proofs, tested geometry, dynamics/refinement, sampling/filtering, probability, measured experiment, and a reproducibility handoff.

Coding slice: Work in `labs/student/lab_10_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Reaching the iteration cap or taking tiny steps does not mean convergence. A constrained optimum need not have zero unconstrained gradient, and error terms cannot be combined with unjustified independence assumptions.

Independent transfer task

For f(x,y)=(x-3)^2+3(y+1)^2, compare alpha=0.1 and alpha=0.4.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Where is the minimum of (x-3)^2?
2. Why can reducing step size help an unstable iteration?
3. What should happen when an iterative method reaches its iteration limit?
4. What range of positive alpha makes the scalar quadratic iteration converge from every finite start in exact arithmetic?

5. Why can x=4 be a constrained optimum even though f prime(4)=2 for f=(x-3)^2?

Assigned study

R42: objectives, convexity, and descent concepts as a bridge; the full graduate optimization treatment comes later. R07: conditioning and least squares. Use the explicit error-factor derivation for this lab.

Original reading assignment retained: R07: least squares and conditioning; R23: numerical utilities; graduate optimization reading in the handbook.

Sources and provenance

What is retained and what is new

The original curriculum and program manifest supplied the scope and lesson sequence. source/original_volume_02.json retains the original Volume 02 definitions; its note identifies original global metadata. New teaching notes, all implementation code, worked exercises and supplementary questions were authored for this bundle. Questions 1-3 in each lesson and the original learning outcomes/assignment briefs are retained from the supplied curriculum. External books and papers are linked, not copied or redistributed as full texts.

The source identifiers below retain the original program's R-codes. Targeted study notes refine navigation without pretending that a single source covers every topic in a broad lesson. In particular, the original 08.4 reading cross-reference to animation through R19 is preserved in the structured source; this bundle derives its oscillator examples directly and directs numerical-method depth to the calculus and numerical-method resources. It does not describe PBRT as a mechanics textbook.

Core MIT course pages for discrete mathematics, linear algebra, single-variable calculus, probability/statistics, signals/systems and multivariable calculus were consulted, along with the Boyd/Vandenberghe book landing page, PBRT transformations and Python math/random documentation, for this September 28, 2026 edition. Other original directory entries are retained as assigned references rather than represented as newly audited in full. Record the chapter/section actually studied and use documentation matching the installed tool. Links, course layouts and software documentation may later change.

Reference directory

R01 — CS2023 knowledge areas

ACM / IEEE-CS / AAAI. Breadth checklist, not a claim of accreditation.

Open the original reference.

R04 — Python tutorial and language documentation

Python Software Foundation. Reference companion; the tutorial itself assumes some programming familiarity.

Open the original reference.

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

Open the original reference.

R07 — 18.06 Linear Algebra

MIT OpenCourseWare. Matrices, vector spaces, eigenvalues and problem sets.

Open the original reference.

R08 — 18.01SC Single Variable Calculus

MIT OpenCourseWare. Calculus lectures, exercises and assessments.

Open the original reference.

R09 — 18.05 Introduction to Probability and Statistics

MIT OpenCourseWare. Probability, inference and quantitative reasoning.

Open the original reference.

R10 — 6.003 Signals and Systems

MIT OpenCourseWare. Sampling, convolution, transforms and systems.

Open the original reference.

R19 — 15-462/662 Computer Graphics

Carnegie Mellon University. Rendering, geometry and animation course structure and exercises.

Open the original reference.

R23 — Physically Based Rendering, fourth edition

Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments.

Open the original reference.

R28 — Box2D documentation

Box2D project. Collision, constraints and simulation reference.

Open the original reference.

R40 — Precalculus, second edition

OpenStax / Rice University. Algebra, functions and trigonometry bridge before calculus.

Open the original reference.

R41 — 18.02SC Multivariable Calculus

MIT OpenCourseWare. Partial derivatives, gradients, multiple integrals and constrained differentials.

Open the original reference.

R42 — Convex Optimization

Stephen Boyd and Lieven Vandenberghe. Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

Open the original reference.

R54 — 18.085 Computational Science and Engineering I

MIT OpenCourseWare. Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

Open the original reference.

Implementation documentation consulted

Python math module and Python random module provide the relevant standard-library contracts. PBRT transformations provides a concrete graphics transformation reference. Moving documentation is not a claim that the tested interpreter is the newest release.

Limits and review

The explanatory derivations and examples are intended for this educational scope. A qualified reviewer should assess written proofs, assumptions, data analysis and independent transfer. External university exercises, academic supervision and production hardware signoff are not replaced by this bundle. All synthetic player/performance data is invented for learning; there is no collected player telemetry. No revenue, employment or research-publication outcome is promised.